THE FAISS LIBRARY

Matthijs Douze
FAIR, Meta

Alexandr Guzhva
Zilliz

Chengqi Deng
DeepSeek

Jeff Johnson
FAIR, Meta

Gergely Szilvasy
FAIR, Meta

Pierre-Emmanuel Mazaré
FAIR, Meta

Maria Lomeli
FAIR, Meta

Lucas Hosseini
Skip Labs

Hervé Jégou
FAIR, Meta

arXiv:2401.08281v4 [cs.LG] 23 Oct 2025

Abstract

Vector databases typically manage large collections of embedding vectors. As AI applications are growing rapidly, the number of embeddings that need to be stored and indexed is increasing. The Faiss library is dedicated to vector similarity search, a core functionality of vector databases. Faiss is a toolkit of indexing methods and related primitives used to search, cluster, compress and transform vectors. This paper describes the trade-offs in vector search and the design principles of Faiss in terms of structure, approach to optimization and interfacing. We benchmark key features of the library and discuss a few selected use cases to highlight its broad applicability.

1 Introduction

The emergence of deep learning has induced a shift in how complex data is stored and searched, noticeably by the development of *embeddings*. Embeddings are vector representations, typically produced by a neural network, that map (embed) the input media item into a vector space, where the locality encodes the semantics of the input. Embeddings are extracted from various forms of media: words [59, 10], text [24, 40], images [15, 69], users and items for recommendation [65]. They can even encode object relations, for instance multi-modal text-image or text-audio relations [31, 70].

Embeddings are employed as an intermediate representation for further processing, e.g. self-supervised image embeddings are input to shallow supervised image classifiers [14, 15]. They are also leveraged as a pretext task for self-supervision [18]. In fact, embeddings are a compact intermediate representation that can be re-used for several purposes.

In this paper, we consider embeddings used directly to compare media items. The embedding extractor is designed so that the distance between embeddings reflects the similarity between their corresponding media. As a result, conducting neighborhood search in this vector space offers a direct implementation of similarity search between media items.

Similarity search is also popular for tasks where end-to-end learning would not be cost-efficient. For example, a k-nearest-neighbor classifier is more efficient to upgrade with new training samples than a classification neural net. This explains why the usage of industrial database management systems (DBMS), that offer a vector storage and search functionality, has increased in the last years. These DBMS are at the junction of traditional databases and Approximate Nearest Neighbor Search (ANNS) algorithms. Until recently, the latter were mostly considered for specific use-cases or in research.

From a practical perspective, the embedding extraction and the vector search algorithm are bound by an "embedding contract" on the embedding distance:

- The embedding extractor, typically a neural network in modern systems, is trained so that distances between embeddings are aligned with the task to perform.
- The vector index performs neighbor search among the embedding vectors as accurately as possible w.r.t. exact search results given the agreed distance metric.

Faiss is a library for ANNS. The core library is a collection of C++ source files without external dependencies. Faiss also provides a comprehensive Python wrapper for its C++ core. It is designed to be used both from simple scripts and as a building block of a DBMS. In contrast with other libraries that focus on a single indexing method, Faiss is a toolbox that contains a variety of indexing methods that commonly involve a chain of components (preprocessing, compression, non-exhaustive search, etc.). In this paper, we show that there exists a choice between a dozen index types, and the optimal one usually depends on the problem's constraints.

To summarize what Faiss is *not*: Faiss does not extract features – it only indexes embeddings that have been extracted by a different mechanism; Faiss is not a service – it only provides functions that are run as part of the calling process on the local machine; Faiss is not a database – it does not provide concurrent write access, load balancing, sharding, transaction management or query optimization. The scope of the library

is intentionally limited to ANNS algorithmic implementation.

The basic structure of Faiss is an *index* that can have multiple implementations described in this paper. An index can store a number of *database vectors* that are progressively added to it. At search time, a *query vector* is submitted to the index. The index returns the database vector that is closest to the query vector w.r.t. the Euclidean distance. There are many variants of this functionality: instead of just the nearest neighbor, k nearest neighbors are returned; instead of a fixed number of neighbors, only the vectors within a certain range are returned; batches of vectors can be searched in parallel; other metrics besides the Euclidean distance are supported; the search can use either CPUs or GPUs.

Since its open-source release in 2017, Faiss has emerged as one of the most popular vector search libraries, boasting 37k GitHub stars and more than 5200 citations of its GPU implementation paper [47]. The Faiss packages have been downloaded 6M times. Major vector database companies, such as Zilliz and Pinecone, either rely on Faiss as their core engine or have reimplemented Faiss algorithms.

This paper exposes the design principles of Faiss. A similarity search library has to trade off between different constraints (Section 3) using two main tools: vector compression (Section 4) and non-exhaustive search (Section 5). We also review a few applications of Faiss for trillion-scale indexing, text retrieval, data mining, and content moderation (Section 7). The appendix discusses how Faiss is structured and engineered to be flexible and usable from other tools. Throughout the paper, we refer to functions or classes in the Faiss codebase[1] as well as the documentation[2] using `this specific style`.

2 Related work

Indexing methods. Over the past decade, a steady stream of papers about indexing methods has been published. Faiss encompasses a broad range of algorithms, catering to a diverse spectrum of use cases.

One of the most popular approaches in the industry is to employ Locality Sensitive Hashing (LSH) as a way to compress embeddings into compact codes. In particular, the cosine sketch [16] produces binary vectors such that the Hamming distance is an estimator of the cosine similarity between the original embeddings. The compactness of these sketches enables storing and searching very large databases of media content [54], without the requirement to store the original embeddings. We refer the reader to the early survey by [91] for research on binary codes.

Since the original work introducing product quantization search [44], quantization-based ANN has emerged as a powerful alternative to binary codes [90]. We refer the reader to the survey by [58] that discusses numerous research works related to quantization-based compact codes.

The term LSH can also refer to indexing with multiple partitions, such as E2LSH [22]. However, we do not consider this scenario as the performance is generally inferior to that of learnt partitions [66]. Early data-aware methods that proved successful on large datasets include multiple partitions based on kd-tree or hierarchical k-means [62]. They are often combined with compressed-domain representation and are especially appropriate for very large-scale settings [43, 44].

After the introduction of NN-descent [26], graph-based ANN algorithms have emerged as a viable alternative to space partitioning methods. Notably, HNSW, currently the most popular indexing method [55] for medium-sized dataset, has been implemented in HNSWlib.

Software packages. Most of the research works on vector search have been open-sourced, and some of these evolved in relatively comprehensive software packages for vector search. For instance, FLANN includes several index types and a distributed implementation described extensively in [62]. The first implementation of product quantization relied on the Yael library [27], that already had a few of the Faiss principles: optimized primitives for clustering methods (GMM and k-means), scripting language interface (Matlab and Python) and benchmarking operators. NMSlib, a package originally designed for text retrieval, was the first to include HNSW [11]. It also provides several index types. The HNSWlib library later became the reference implementation of HNSW [55]. Google's SCANN library is a thoroughly optimized implementation of IVFPQ [44] on SIMD and includes several index variants for various database scales. SCANN was open-sourced but the accompanying paper [36] omits to mention the engineering optimizations that underpin the library's remarkable speed. DiskANN [79] is Microsoft's foundational graph-based vector search library. It was originally built to leverage hybrid RAM/flash memory, but now offers a RAM-only version and was later extended to perform efficient updates [78], out-of-distribution search [42] and filtered search [34].

Faiss was open-sourced concurrently with a publication [47] that details the GPU implementation of several index types. The present paper complements this previous work by describing the library as a whole.

Concurrently, various software libraries from the database world were extended or developed to do vector search. Milvus [89] uses its Knowhere library, which relies on Faiss as one of its core engines. Pinecone [12] initially relied on Faiss, although the engine was later rewritten in Rust. Weaviate [86] is a composite retrieval engine that includes vector search among other methods.

[1] https://github.com/facebookresearch/faiss
[2] https://faiss.ai/

Theoretical guarantees. Several approximate algorithms were inspired by the Johnson-Lindenstrauss lemma, which stipulates that one can embed a set of high-dimensional points into a lower-dimensional space while almost preserving the distances. For instance, under certain assumptions about the data distribution, LSH algorithms based on random partitioning (e.g. projection) provide statistical guarantees for the range search problem. As this objective is only a proxy to the problem of nearest neighbor search, there are no formal guarantees w.r.t. any metrics such as nearest-neighbor recall. In practice, algorithms based on random partitioning are significantly outperformed by data-aware partitioning [66, 2]. Similarly, graph-based algorithms are difficult to compare from a theoretical perspective if the data distribution is unknown.

Techniques based on vector compression, either with binary sketches or quantization, often correspond to estimators of the target distance. For some of these methods, the variance of the estimator can be computed in closed form [16, 44], characterizing the distance estimation error. Again, the quality of the estimator is only a proxy for the actual k-nearest-neighbor problem.

Considering these factors and the importance of execution speed, the quality of ANN search is typically evaluated by experimental comparisons on publicly available benchmarks.

Benchmarks and competitions. The leading benchmark for million-scale datasets is ANN-benchmarks [3]. It compares about 50 implementations of ANNS. The big-ANN [76] challenge introduced large-scale settings, with 6 datasets containing 1 billion vectors each. Faiss was used as a baseline for the challenge and multiple submissions were derived from it. The 2023 edition of the challenge is at a smaller scale (10M vectors) but introduces more complex tasks, including a filtered track where Faiss served as a baseline method [75].

Datasets. Early datasets are based on keypoint features like SIFT [53] used in image matching. We use BIGANN [46], a dataset of 128-dimensional SIFT features. Later, when global image descriptors produced by neural nets became popular, the Deep1B dataset was released [6], with 96-dimensional image features extracted with Google LeNet [82]. For this paper, we introduce a dataset of 768-dimensional Contriever text embeddings [40] that are compared using inner product similarity. The embeddings are computed with English Wikipedia passages. The higher dimension of these embeddings is representative of contemporary applications.

Each dataset has 10k query vectors and 20M to 350M training vectors. We indicate the size of the database explicitly, for example "Deep1M" means the database contains the 1M first vectors of Deep1B. The training, database and query vectors are sampled randomly from the same distribution: we do not address out-of-distribution data [42, 8] in this work.

Table 1: Common notations used throughout the paper.

Notation	Meaning
d	vector dimension
N	number of vectors
$q \in \mathbb{R}^d$	query vector
$x_i \in \mathbb{R}^d$	i^{th} database vector
k	number of requested results
$\varepsilon \in \mathbb{R}^+$	radius for range search
K	number of centroids for quantization
M	number of sub-quantizers

3 Performance axes of a vector search library

Vector search is a well-defined, unambiguous operation. In its simplest formulation, given a set of database vectors $\{x_i, i = 1..N\} \subset \mathbb{R}^d$ and a query vector $q \in \mathbb{R}^d$, it computes

$$n = \underset{i=1..N}{\text{argmin}} \|q - x_i\|. \tag{1}$$

The minimum can be computed with a direct algorithm by iterating over all database vectors: this is *brute force search*. A slightly more general and complex operation is to compute the k nearest neighbors of q:

$$(n_1, ..., n_k, *, ..., *) = \underset{i=1..N}{\text{argsort}} \|q - x_i\|, \tag{2}$$

where argsort returns the indices of the array to sort it by increasing distances and $*$ means that an output is ignored. This is what the `search` method of a Faiss index returns. A related operation is to find all the elements that are within some ε distance to the query:

$$R = \{n = 1..N \text{ s.t. } \|q - x_n\| \leq \varepsilon\}, \tag{3}$$

which is computed with the `range_search` method.

Distance measures. In the equations above, we leave the definition of the distance undefined. Faiss aims at supporting a range of distance metrics, but in order to avoid overloading the library, it leaves out metrics that are equivalent up to a monotonous transformation of distances (like Euclidean vs. squared Euclidean) or up to a preprocessing of the input vectors (like Mahalanobis or cosine, see below). The full list of metrics is in Appendix A.6.

The most commonly used distances in Faiss are the L2 distance, the cosine similarity and the inner product similarity (for the latter two, the argmin should be replaced with an argmax). These measures have useful analytical properties: for example, they are invariant under d-dimensional rotations.

In terms of applications, the Euclidean distance is the default for comparing vectors [53]. When vectors are obtained by metric learning, the cosine similarity is often used to avoid collapsing or exploding embeddings [23]. Maximum inner product search (MIPS) is most often used for recommendation systems that require to compare user and item embeddings [65].

These measures can be made equivalent by preprocessing transformations on the query and/or the database vectors. Table 2 summarizes the preprocessing transformations mapping different measures. To our knowledge, some of these were already identified [7, 37], while others are new.

Note that vectors transformed in this manner have a very anisotropic distribution [60]: the additional dimension incurred for many transformations is not homogeneous w.r.t. other dimensions. This can make indexing more difficult, in particular using product or scalar quantization methods. See Section 4.2 for mitigations.

3.1 Brute force search

Developing an efficient implementation of brute force search is not trivial [21, 47]. It requires (1) an efficient way of computing the distances and (2) an efficient way of keeping track of the k smallest distances.

Computing distances in Faiss is performed either by direct distance computations, or, when query vectors are provided in large enough batches, using a matrix multiplication decomposition [47, Equation 2]. The corresponding Faiss functions are exposed in `knn` and `knn_gpu` for CPU and GPU, respectively.

Collecting the top-k smallest distances is usually done via a binary heap on CPU [27, section 2.1] or a sorting network on GPU [47, 63]. For larger values of k, it is more efficient to use a reservoir: an unordered result buffer of size $k' > k$ that is resized to k when it overflows.

Faiss's `IndexFlat` implements brute force search. However, for large datasets this approach becomes too slow. In low dimensions, there are branch-and-bound methods that yield exact search results. However, in large dimensions they provide no speedup over brute force search [93].

Besides, for some applications, small variations in distances are not significant enough to distinguish application-level positive and negative items [83]. In these cases, approximate nearest neighbor search (ANNS) becomes interesting.

3.2 Metrics for Approximate Nearest Neighbor Search

With ANNS, the user accepts imperfect results, which opens the door to a new solution design space. With exact search, the database is represented as a plain matrix. For ANNS, the database may be preprocessed into an indexing structure, more simply referred to as an *index* in the following.

Accuracy metrics. In ANNS the **accuracy**[3] is measured as a discrepancy with the exact search results from (2) and (3). Note that this is an intermediate goal: the end-to-end accuracy depends on (1) how well the distance metric correlates with the item matching objective and (2) the quality of ANNS, which is what we measure here.

Accuracy of k-nearest neighbor search is generally evaluated with "n-recall@k", which is the fraction of the n ground-truth nearest neighbors that are in the k first search results (with $n \leq k$). Most often $k = 1$ or $n = k$ (in which case the measure is a.k.a. "intersection measure"). When $n = k = 1$, the recall measure and intersection are the same, and the recall is called "accuracy". In some publications [44], recall@k means 1-recall@k, while in others [77] it corresponds to k-recall@k.

For range search, the exact search result is obtained by applying (3), using threshold ε. To yield result list $\widehat{R}$, the approximate search uses a (possibly different) threshold ε'. Thus, standard retrieval metrics can be computed: **precision** $P = |R \cap \widehat{R}|/|\widehat{R}|$ and **recall** $R = |R \cap \widehat{R}|/|R|$. By sweeping ε' from small to large, the result list $\widehat{R}$ increases, producing a precision-recall curve. The area under the PR-curve is the **mean average precision** score of range search [76]. Setting $\varepsilon' \neq \varepsilon$ is relevant when approximate vector representations distort the distance metric (more on this in Section 4).

For vector encoder-decoder pairs, the standard metric is the mean squared error (MSE) between the original vector and the reconstructed vector [44, 4, 39]. For an encoder C and a decoder D, the MSE is:

$$\mathrm{MSE} = \mathbb{E}_x\left[\|D(C(x)) - x\|_2^2\right]. \tag{4}$$

Resource metrics. The other axes of the trade-off are related to computing resources. During search, the **search time** and **memory usage** are the main constraints. If compression is used, then, the memory usage can be smaller than what is required to store the original vectors.

The index may need to store training data, which incurs a **constant memory overhead** before any vector is added to the index. The index can also add **per-vector memory overhead** to the memory used to store each vector. This is the case for graph indexes, that need to store graph edges for each vector. The memory usage is more complex for settings with hybrid storage such as RAM + flash or GPU memory + RAM.

The **index building time** is also a resource constraint. It may be decomposed into a **training time**, which is independent of the number of vectors added to the index, and the **addition time per vector**.

In distributed settings or flash-backed storage, the relevant metric is the **number of I/O operations** (IOPS), as each read operation fetches a whole page. Data layouts that minimize IOPs [79] are more efficient than small random accesses. Another possibly

[3] Metrics involved in a tradeoff are indicated in a **specific font**.

Table 2: One wants to compare a query vector x and a database vector y given a particular metric (rows). The table indicates how to preprocess $(x, y) \longmapsto (x', y')$ so that an index with another metric (columns) returns the nearest neighbors for the source metric. Some cases require adding 1 or 2 extra dimensions to the original vectors, as is denoted by the vector concatenation symbol $[.;.]$. The positive scalar parameters α and β are arbitrary and chosen to avoid negative values under a square root.

index metric → wanted metric ↓	L2	IP	cos
L2	identity	$x' = [x; -\alpha/2]$ $y' = [y; \|y\|^2/\alpha]$	$x' = [x; -\alpha/2; 0]$ $y' = [\beta y; \beta\|y\|^2/\alpha;$ $\sqrt{1 - \beta^2\|y\|^2 - \beta^2\|y\|^4/\alpha^2}]$
IP	$x' = [x; 0]$ $y' = [y; \sqrt{\alpha^2 - \|y\|^2}]$	identity	$x' = [x; 0]$ $y' = [\alpha y; \sqrt{1 - \|\alpha y\|^2}]$
cos	$x' = x/\|x\|$ $y' = y/\|y\|$	$x' = x/\|x\|$ $y' = y/\|y\|$	identity

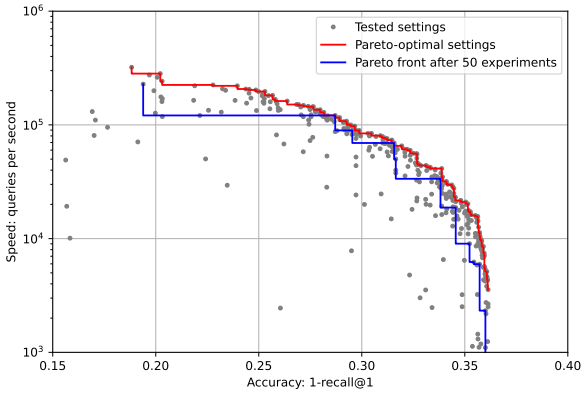


Figure 1: Example of exploration of a parameter space with 3 parameters (an `IndexIVFPQ` with polysemous codes and HNSW coarse quantizer, running on the Deep100M dataset). The total number of configurations is 5808, but only 398 experiments are run. We also show the set of operating points obtained with just 50 experiments.

limiting factor is the amount of **extra memory** needed to pre-compute lookup tables used to speed up search.

3.3 Tradeoffs

Most often only a subset of metrics matter. For example, when a very large number of searches are performed on a fixed index, the **index building time** does not matter. Or when the number of vectors is so small that the raw database fits in RAM multiple times, then the **memory usage** does not matter. We refer to the metrics that we care about as the *active constraints*. Note that **accuracy** is always an active constraint because if it did not matter, returning random results would be sufficient (Faiss does actually provide an `IndexRandom` used in some benchmarking tasks).

In the following sections, we consider that the active constraints are **speed**, **memory usage** and **accuracy**. As such, we measure the **speed** and **accuracy** of several index types and hyperparameter settings under a fixed memory budget.

3.4 Exploring search-time settings

For a fixed index, there are often one or several search-time hyperparameters that trade off speed with accuracy. For example, the `nprobe` hyperparameter for an `IndexIVF`, see Section 5. In general, we define hyperparameters as discrete scalar values such that when the value is higher, the **speed** decreases and the **accuracy** increases. We can then keep only the Pareto-optimal settings, defined as settings that are the fastest for a given accuracy, or equivalently, that have the highest accuracy for a given time budget [80].

Exploring the Pareto-optimal frontier when there is a single hyper-parameter consists in sweeping over its values with a certain level of granularity, and measuring the corresponding **speed** and **accuracy**.

For multiple hyperparameters, the Pareto frontier can be recovered by exhaustively testing the Cartesian product of these parameters. However, the number of settings to test grows exponentially with the number of parameters.

Pruning the parameter space. one can leverage the monotonous nature of the hyperparameters for efficient pruning. We note a tuple of n hyper-parameters $\pi = (p_1, ..., p_n) \in \mathcal{P} = \mathcal{P}_1 \times ... \times \mathcal{P}_n$, and $\leq$ a partial ordering on $\mathcal{P}$: $(p_1, .., p_n) \leq (p'_1, .., p'_n) \Leftrightarrow \forall i, p_i \leq p'_i$. Let $S(\pi)$ and $A(\pi)$ be the speed and accuracy obtained with this tuple of parameters. $\mathcal{P}^* \subset \mathcal{P}$ is the set of Pareto-optimal settings:

$$\mathcal{P}^* = \left\{\pi \in \mathcal{P} | \nexists \pi' \in \mathcal{P} \text{ s.t. } (S(\pi'), A(\pi')) > (S(\pi), A(\pi))\right\}. \quad (5)$$

Since the individual parameters have a monotonic effect on speed and accuracy, we have

$$\pi' \geq \pi \Rightarrow \begin{cases} S(\pi') \leq S(\pi), \\ A(\pi') \geq A(\pi). \end{cases} \quad (6)$$

Thus, if a subset $\widehat{\mathcal{P}} \subset \mathcal{P}$ of settings is already evaluated, the following upper bounds hold for a new setting $\pi \in \mathcal{P}$:

$$S(\pi) \leq \widehat{S}(\pi) = \inf_{\pi' \in \widehat{\mathcal{P}} \text{ s.t. } \pi' \leq \pi} S(\pi'), \quad (7)$$

$$A(\pi) \leq \widehat{A}(\pi) = \inf_{\pi' \in \widehat{\mathcal{P}} \text{ s.t. } \pi' \geq \pi} A(\pi'). \quad (8)$$

If any previous evaluation Pareto-dominates these bounds, the setting π does not need to be evaluated:

$$\exists \pi' \in \widehat{\mathcal{P}} \text{ s.t. } (S(\pi'), A(\pi')) > (\widehat{S}(\pi), \widehat{A}(\pi)) \Rightarrow \pi \notin \mathcal{P}^*. \quad (9)$$

In practice, we evaluate settings from $\mathcal{P}$ in a random order. The pruning becomes more and more effective throughout the process. It is also more effective when the number of parameters is larger. Figure 1 shows an example with $|\mathcal{P}| = 5808$ combined parameter settings. The pruning from (9) reduces this to 398 experiments, out of which $|\mathcal{P}^*| = 87$ are optimal. The Faiss `OperatingPoints` object implements this pruning.

3.5 Refining (`IndexRefine`)

One can combine a fast but inaccurate index with a slower, more accurate search. [46, 79, 36]. This is done by querying the fast index to retrieve a shortlist of results. The more accurate search then computes more accurate results only for the shortlist. This requires the accurate index to allow efficient random access to database vectors. Some implementations use a slower storage (e.g. flash) for the second index [79, 81].

For the first-level index, the relevant accuracy metric is the recall at a rank equal to the shortlist size. Thus, 1-recall@1000 can be a relevant metric, even if the end application does not use the 1000th neighbor.

Several methods based on this refining principle do not use two separate indexes. Instead, they use two ways of interpreting the same compressed vectors: a fast and inaccurate decoding and a slower but more accurate decoding [28, 29, 61, 1, 39] are based on this principle. The polysemous codes method [28] is implemented in Faiss's `IndexIVFPQ`.

4 Compression levels

Faiss supports various vector codecs: these are methods to compress vectors so that they take up less memory. A compression method $C : \mathbb{R}^d \to \{1, ..., K\}$, a.k.a. a quantizer, converts a continuous multi-dimensional vector to an integer. This integer is equivalent to a bit string of code size $\lceil \log_2 K \rceil$. The decoder $D : \{1, ..., K\} \to \mathbb{R}^d$ reconstructs an approximation of the vector from the integer. The decoder can only reconstruct a finite number, K, of distinct vectors.

The search of (1) becomes approximate:

$$n = \underset{i=1..N}{\operatorname{argmin}} \|q - D(C(x_i))\| = \underset{i=1..N}{\operatorname{argmin}} \|q - D(C_i)\|, \quad (10)$$

where the codes $C_i = C(x_i)$ are precomputed and stored in the index. This is the asymmetric distance computation (ADC) [44]. The symmetric distance computation (SDC) corresponds to the case when the query vector is also compressed:

$$n = \underset{i=1..N}{\operatorname{argmin}} \|D(C(q)) - D(C_i)\|. \quad (11)$$

Most Faiss indexes perform ADC as it is more accurate: no accuracy is lost on the query vectors. SDC is useful when there is also a storage constraint on the queries or for indexes where SDC is faster to compute than ADC. The naive computation of (10) decompresses the vectors, which has an impact on speed. In most cases, the distance can be computed in the compressed domain.

4.1 The vector codecs

The k-means vector quantizer (`Kmeans`). The ideal vector quantizer minimizes the MSE between the original and the decompressed vectors. This is formalized in the Lloyd necessary conditions for the optimality of a quantizer [52].

The k-means algorithm directly implements these conditions. The K centroids of k-means are an explicit enumeration of all possible vectors that can be reconstructed.

The k-means vector quantizer is very **accurate** but the **memory usage** and **encoding complexity** grow exponentially with the code size. Therefore, k-means is impractical to use beyond roughly 3-byte codes, corresponding to 16M centroids.

Scalar quantizers. Scalar quantizers encode each dimension of a vector independently.

A very classical and simple scalar quantizer is LSH (`IndexLSH`), where each vector component is encoded in a single bit by comparing it to a threshold. The threshold can be fixed to 0 or trained. Faiss supports efficient SDC search of binary vectors via the `IndexBinary` objects, see Section 4.5.

The `ScalarQuantizer` also supports uniform quantizers that encode a vector component into 8, 6 or 4 bits – referred to as `SQ8`, `SQ6`, `SQ4`. A scale and offset determine which values are reconstructed. They can be set separately for each dimension on the whole vector. The `IndexRowwiseMinMax` stores vectors with per-vector normalizing coefficients. Lower-precision 16-bit floating point representations are also considered as scalar quantizers, `SQfp16` and `SQbf16`.

Multi-codebook quantizers. Faiss contains several multi-codebook quantization (MCQ) options. They are built from M vector quantizers that can reconstruct K distinct values each. The codes produced by these methods are of the form $(c_1, ..., c_M) \in \{1, ..., K\}^M$, i.e. each code indexes one of the quantizers. The number of reconstructed vectors is K^M and the code size is thus $M\lceil \log_2(K) \rceil$.

The product quantizer (`ProductQuantizer`, also noted PQ) is a simple MCQ that splits the input vector into M sub-vectors and quantizes them separately [44] with a k-means quantizer. At reconstruction time, the individual reconstructions are concatenated to produce the final code. In the following, we will use the notation `PQ6x10` for a product quantizer with 6 sub-vectors each encoded in 10 bits ($M = 6$, $K = 2^{10}$).

Additive quantizers are a family of MCQ where the reconstructions from sub-quantizers are summed up together. Finding the optimal encoding for a vector given the codebooks is NP-hard [4], so, in practice, additive quantizers use heuristics to find near-optimal codes.

Faiss supports two types of additive quantizers. The residual quantizer (`ResidualQuantizer`) proceeds sequentially, by encoding the difference (residual) of the vector to encode and the one that is reconstructed by the previous sub-quantizers [19]. The local search quantizer (`LocalSearchQuantizer`) starts from a sub-optimal encoding of the vector and locally explores neighbording codes in a simulated annealing process [56, 57]. We use notations `LSQ6x10` and `RQ6x10` to refer to additive quantizers with 6 codebooks of size 2^{10}.

Faiss also supports a combination of PQ and additive quantizer, `ProductResidualQuantizer`. In that case, the vector is split in sub-vectors that are encoded independently with additive quantizers [5]. The codes from the sub-quantizers are concatenated. We use the notation `PRQ2x6x10` to indicate that vectors are split in 2 and encoded independently with `RQ6x10`, yielding a total of 12 codebooks of size 2^{10}.

Hierarchy of quantizers. Although this is not by design, there is a strict ordering between the quantizers described before. This means that quantizer $i+1$ can have the same set of reproduction values as quantizer i: it is more flexible and more data adaptive. The hierarchy of quantizers is:

1. the binary representation with bits +1 and -1 can be represented as a scalar quantizer with 1 bit per component;
2. the scalar quantizer is a product quantizer with 1 dimension per sub-vector and uniform per-dimension quantizer;
3. the product quantizer is a product-additive quantizer where the additive quantizer has a single level;
4. the product additive quantizer is an additive quantizer where within each codebook all components outside one sub-vector are set to 0 [4];
5. the additive quantizer is the general case where the codebook entries correspond to all possible reconstructions obtained by adding elements from the subquantizers.

The implications of this hierarchy are (1) the degrees of freedom for the reproduction values of quantizer $i+1$ are larger than for i, so it is more **accurate** (2) quantizer $i+1$ has a higher capacity so it consumes more resources in terms of **training time** and **storage overhead** than i. In practice, the product quantizer often offers a good trade-off, which explains its wide adoption. The corresponding Faiss `Quantizer` objects are listed in Appendix A.7.

4.2 Vector preprocessing

Applying transformations to input vectors before encoding can enhance the effectiveness of certain quantizers. In particular, d-dimensional rotations are commonly used, as they preserve comparison metrics like cosine, L2 and inner product.

Scalar quantizers assign the same number of bits per vector component. However, for distance comparisons, if specific vector components have a higher variance, they have more impact on the distances. In other works, a variable number of bits are assigned per component [71]. However, it is simpler to apply a random rotation to the input vectors, which in Faiss can be done with a `RandomRotationMatrix`. The random rotation spreads the variance over all the dimensions without changing the measured distances.

An important transform is the Principal Component Analysis (PCA), that reduces the number of dimensions d of the input vectors to a user-specified d'. This operation (`PCAMatrix`) is the orthogonal linear mapping that best preserves the variance of the input distribution. It is often beneficial to apply PCA to large input vectors before quantizing them as k-means quantizers are more likely to "fall" in local minima in high-dimensional spaces [51, 45].

The OPQ transformation [33] is a rotation of the input space that decorrelates the distribution of each sub-vector of a product quantizer[4]. This makes PQ more accurate in the case where the variance of the data is concentrated on a few components. The Faiss implementation `OPQMatrix` combines OPQ with a dimensionality reduction. The ITQ transformation [35] similarly rotates the input space prior to binarization (`ITQMatrix`).

4.3 Faiss additive quantization options

Additive quantizers exist in two main variants: the residual quantizer and local search quantizer. They are more complex than most quantizers because the **index building time** must be taken into account. In fact, the accuracy of an additive quantizer of a certain size can always be increased at the cost of an increased encoding time (and training time).

Additive quantizers are based on M codebooks $T_1, ...T_M$ of size K. The decoding of code $C(x) = (c_1, ..., c_M)$ is

$$x' = D(C(x)) = T_1[c_1] + ... + T_M[c_M]. \quad (12)$$

Thus, decoding is unambiguous. However, there is no practical way to encode vectors optimally, let alone train the codebooks. Enumerating all possible encodings is of exponential complexity in M.

The residual quantizer (RQ). RQ encodes a vector x sequentially. At stage m, RQ picks the entry that

[4] In Faiss terms, OPQ and ITQ are preprocessings. The actual quantization is performed by a subsequent product quantizer or binarization step.

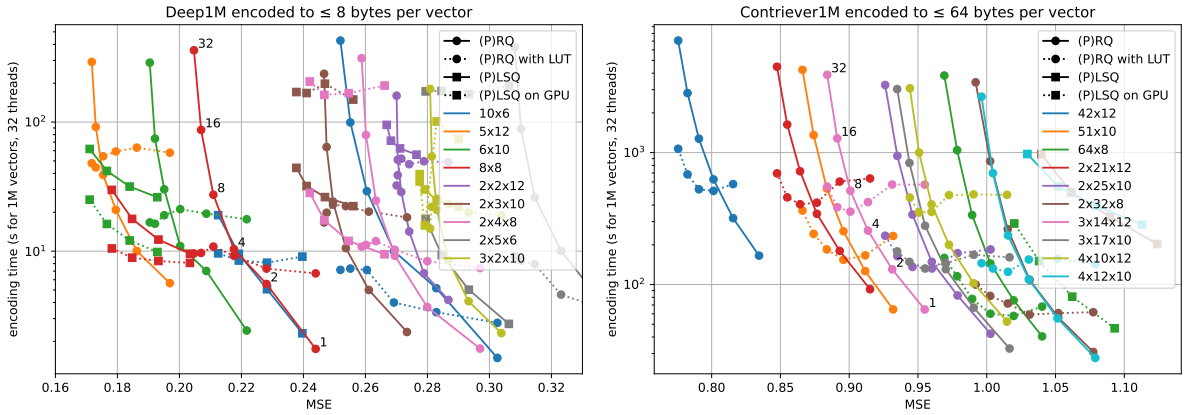


Figure 2: Comparison of additive quantizers in terms of **encoding time** vs. **accuracy** (MSE). Lower values are better for both. We consider two different regimes: Deep1M (low-dimensional) to 8-bytes codes and Contriever1M (high dimensional) to 64-byte codes. For some RQ variants, we indicate the beam size setting at which that trade-off was obtained.

best reconstructs the residual of x w.r.t. the previous encoding steps:

$$c_m = \underset{j=1..K}{\operatorname{argmin}} \left\| \sum_{i=1}^{m-1} T_i[c_i] + T_m[j] - x \right\|^2. \quad (13)$$

This greedy approach tends to get trapped in local minima. As a mitigation, the encoder maintains a beam of `max_beam_size` of possible codes and picks the best code at stage M. This parameter adjusts the trade-off between **encoding time** and **accuracy**.

To speed up the encoding, the norm of (13) can be decomposed into the sum of:

- $\|T_m[j]\|^2$ is precomputed and stored;
- $\left\|\sum_{i=1}^{m-1} T_i[c_i] - x\right\|^2$ is the encoding error of the previous step $m-1$;
- $-2\langle T_m[j], x\rangle$ is computed on entry to the encoding (it is the only computation complexity that depends on d);
- $2\sum_{\ell=1}^{m-1}\langle T_m[j], T_\ell[c_\ell]\rangle$ is also precomputed.

This decomposition is used when `use_beam_LUT` is set. It is interesting only if d is large and when M is small because the storage and compute requirements of the last term grow quadratically with M.

The local search quantizer (LSQ). At encoding time, LSQ starts from a suboptimal encoding of the vector and proceeds with a simulated annealing optimization to refine the codes. At each optimization step, LSQ randomly flips codes and then uses Iterated Conditional Mode (ICM) to optimize the new encoding. The number of optimization steps is set with `encode_ils_iters`. The LSQ codebooks are trained via an expectation-maximization procedure (similar to k-means).

Compressed-domain search. The distances are computed without decompressing the stored vectors. It is acceptable to perform pre-computations on the query vector q because the cost of these pre-computations is amortized over many query-to-code distance comparisons.

Additive quantizer inner products can be computed in the compressed domain:

$$\langle q, x'\rangle = \sum_{m=1}^{M} \langle T_m[c_m], q\rangle = \sum_{m=1}^{M} \mathrm{LUT}_m[c_m]. \quad (14)$$

The lookup tables LUT_m are computed when a query vector comes in, similar to product quantizer search [44].

This decomposition does not work to compute L2 distances. As a workaround, Faiss uses the decomposition [4]

$$\|q - x'\|^2 = \|q\|^2 + \|x'\|^2 - 2\langle q, x'\rangle. \quad (15)$$

Thus, the term $\|x'\|^2$ must be available at search time. Using the `AdditiveQuantizer.search_type` configuration, it can be appended in the stored code (`ST_norm_float32`), possibly compressed (`ST_norm_qint8`, `ST_norm_qint4`,...). It can also be computed on the fly (`ST_norm_from_LUT`) with

$$\|x'\|^2 = 2\sum_{m=1}^{M}\sum_{\ell=1}^{m-1}\langle T_m[c_m], T_\ell[c_\ell]\rangle + \sum_{m=1}^{M}\|T_m[c_m]\|^2. \quad (16)$$

There, the norms and dot products are stored in the same lookup tables as the one used for beam search. Therefore, it trades off **search time** for **memory** overhead to store codes.

Figure 2 shows the trade-off between **encoding time** and **MSE**. Given a code size, it is more accurate to use a smaller number of sub-quantizers M and a higher K. GPU encoding for LSQ does not help systematically. The LUT-based encoding of RQ is interesing for RQ/PRQ quantization when the beam size is larger. In the 64-byte regime, we observe that

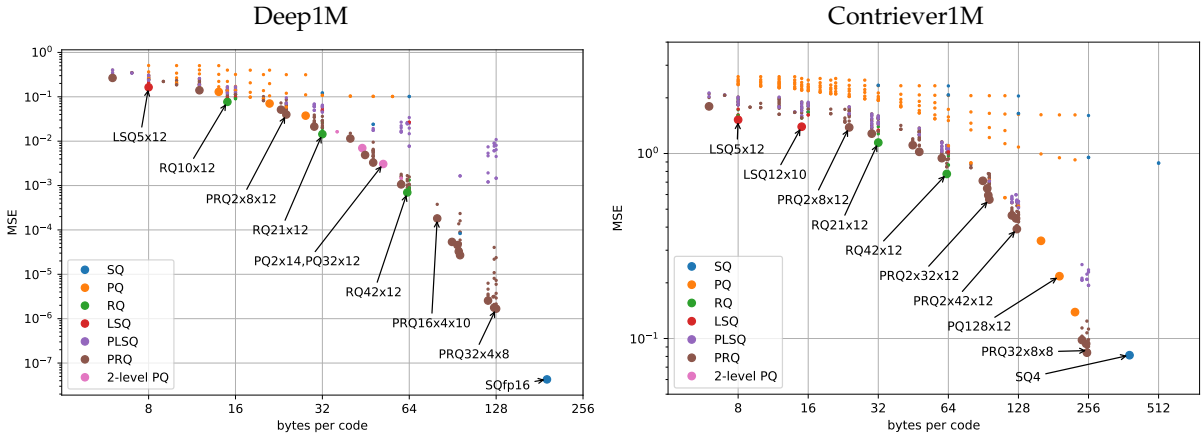


Figure 3: **Accuracy** vs. **code size** trade-off for different codecs on the Deep1M and Contriever1M datasets. We show Pareto-optimal variants with larger dots and indicate the quantizer in text for some of them. Note that contriever vectors can be encoded to MSE=$2 \cdot 10^{-4}$ in 768 bytes with `SQ8` (that setting is widely out-of-range for the plot).

LSQ is not competitive with RQ. PLSQ and PRQ progressively become more competitive for larger memory budgets. They are also faster, since they operate on smaller vectors.

4.4 Vector compression benchmark

Figure 3 shows the trade-off between **code size** and **accuracy** for many variants of the codecs. Additive quantizers are the best options for small code sizes. For larger code sizes, it is beneficial to independently encode several sub-vectors with product-additive quantizers. LSQ is more accurate than RQ for small codes, but does not scale well to longer codes. Note that product quantizers are a bit less accurate than additive quantizers, but given their low **encoding time** they remain an attractive option. The scalar quantizers perform well for very long codes and are even faster. The 2-level PQ options are what an IVFPQ index uses as encoding: a first-level coarse quantizer and a second level refinement of the residual (more about this in Section 5.1).

4.5 Binary indexes

Binary quantization with symmetric distance computations is a pattern that has been commonly used [91, 13]. In this setup, distances are computed in the compressed domain as Hamming distances. (11) reduces to:

$$n = \underset{i=1..N}{\operatorname{argmin}} \|C(q) - C_i\|. \tag{17}$$

where $C(q), C_i \in \{0,1\}^d$. Hamming distances are integers in $\{0..d\}$. Although they are crude approximations for continuous domain distances, they are fast to compute, do not require any specific context, and are easy to calibrate in practice.

The `IndexBinary` indexes support addition and search directly from binary vectors. They offer a compact representation and leverage optimized instructions for distance computations.

The simplest `IndexBinaryFlat` index performs exhaustive search. Three options are offered for non-exhaustive search:

- `IndexBinaryIVF` is a binary counterpart for the inverted-list `IndexIVF` index described in 5.1.
- `IndexBinaryHNSW` is a binary counterpart for the hierarchical graph-based `IndexHNSW` index described in 5.2.
- `IndexBinaryHash` uses prefix vectors as hashes to cluster the database (rather than spheroids as with inverted lists), and searches only the clusters with closest prefixes.

Finally, the`IndexBinaryFromFloat` is provided for convenience. It wraps an arbitrary index and offers a binary vector interface for its operations.

5 Non-exhaustive search

Non-exhaustive search is the cornerstone of fast search implementations for datasets larger than around N=10k vectors. In that case, the aim of the indexing method is to quickly focus on a subset of database vectors that are most likely to contain the search results.

A method to do this is Locality Sensitive Hashing (LSH). It amounts to projecting the vectors on a random direction [22]. The offsets on that direction are then discretized into buckets where the database vectors are stored. At search time, only the nearest buckets to the query vector's projection are visited. In practice, *several* projection directions are needed to make it **accurate**, at the cost of **search time** and **memory usage**. A fundamental drawback of this method is that it is not data-adaptive, although some improvements are possible [66].

An alternative way of pruning the search space is to use tree-based indexing. In that case, the dataset is stored in the leaves of a tree [62]. When querying a

vector, the search starts at the root node. At each internal node, the search descends into one of the child nodes depending on a decision rule. The decision rule depends on how the tree was built: for a KD-tree it is the position w.r.t. a hyperplane, for a hierarchical k-means, it is the proximity to a centroid.

LSH and tree-based methods both aim to extend classical database search structures to vector search, because they have a favorable complexity (constant or logarithmic in N). However, these methods do not scale well for dimensions above 10.

Faiss implements two non-exhaustive search approaches that operate at different **memory** vs. **speed** trade-offs: inverted file and graph-based.

5.1 Inverted files

IVF indexing is a technique that clusters the database vectors at indexing time. This clustering uses a vector quantizer (the *coarse quantizer*) that outputs K_{IVF} distinct indices (the `nlist` field of the `IndexIVF` object). The coarse quantizer's K_{IVF} reproduction values are called *centroids*. The vectors of each cluster (possibly compressed) are stored contiguously into inverted lists, forming an inverted file (IVF). At search time, only a subset of P_{IVF} clusters are visited (a.k.a. `nprobe`). The subset is formed by searching the P_{IVF} nearest centroids, as in (2).

Setting the number of lists. The K_{IVF} parameter is central. In the simplest case, when P_{IVF} is fixed, the coarse quantizer is exhaustive, the inverted lists contain uncompressed vectors, and the inverted lists are all the same size, then the number of distance computations is

$$N_{\mathrm{distances}} = K_{\mathrm{IVF}} + P_{\mathrm{IVF}} \times N/K_{\mathrm{IVF}} \qquad (18)$$

reaching a minimum when $K_{\mathrm{IVF}} = \sqrt{P_{\mathrm{IVF}} N}$. This yields the usual recommendation to set P_{IVF} proportional to $\sqrt{N}$.

In practice, this is just a rough approximation because (1) the P_{IVF} has to increase with the number of lists in order to keep a fixed **accuracy** (2) the inverted lists sizes are not balanced (3) often the coarse quantizer is not exhaustive itself, so the quantization uses fewer than K_{IVF} distance computations, for example it is common to use a non-exhaustive HNSW index to perform the coarse quantization.

The *imbalance factor* is the relative variance of inverted list sizes [84]. At search time, if the inverted lists all have the same length, this factor is 1. If they are unbalanced, the expected number of distance computations is multiplied by this factor.

Figure 4 shows the optimal settings of K_{IVF} for various database sizes. For a small $K_{\mathrm{IVF}} = 4096$, the coarse quantization runtime is negligible and the **search time** increases linearly with the database size. On larger datasets, it is beneficial to increase K_{IVF}. As in (18), the ratio $K_{\mathrm{IVF}}/\sqrt{N}$ is roughly 15 to 20. Note that this ratio depends on the data distribution and

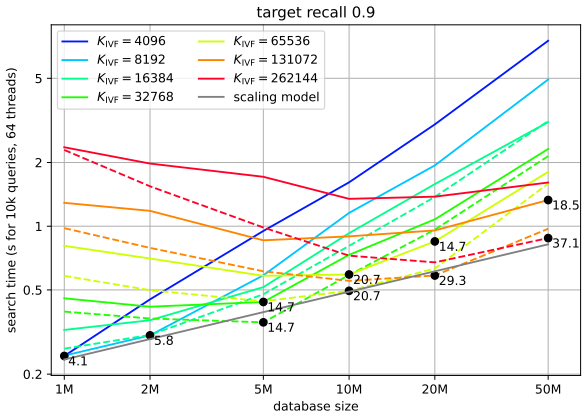


Figure 4: **Search time** as a function of the database size N for BigANN1B with different K_{IVF} settings. The P_{IVF} is set so that the **1-recall@1** is 90%. The full lines indicate that the coarse quantizer is exact, the dashed lines rely on a HNSW coarse quantizer. For some setting we indicate the ratio $K_{\mathrm{IVF}}/\sqrt{N}$

the target **accuracy**. Interestingly, in a regime where K_{IVF} is larger than the optimal setting for N (e.g. $K_{\mathrm{IVF}} = 2^{18}$ and N =5M), the P_{IVF} needed to reach the target accuracy *decreases* with the dataset size, and so does the **search time**. This is because when K_{IVF} is fixed and N increases, for a given query vector, the nearest database vector is either the same or a new one that is closer, so it is more likely to be found in a quantization cluster nearer to the query.

With a faster non-exhaustive coarse quantizer (e.g. HNSW) it is even more useful to increase K_{IVF} for larger databases, as the coarse quantization becomes relatively cheap. At the limit, when $K_{\mathrm{IVF}} = N$, then all the work is done by the coarse quantizer. In this scenario, the limiting factor becomes the **memory overhead** of the coarse quantizer.

By fitting a model of the form $t = t_0 N^{\alpha}$ to the timings of the fastest index in Figure 4, we can derive a scaling rule for the IVF indexes:

target recall@1	0.5	0.75	0.9	0.99
power α	0.29	0.30	0.34	0.45

Thus, with this model, the search time increases faster for higher accuracy targets, but $\alpha < 0.5$, so the runtime dependence on the database size is below $\sqrt{N}$. This empirical complexity analysis has been used before for graph-based indices [92].

Encoding residuals. In general, it is more **accurate** to compress the residuals of the database vectors w.r.t. the centroids [44, Eq. (28)]. This is either because the norm of the residuals is lower than that of the original vectors, or because residual encoding is a way to take into account a-priori information from the coarse quantizer. In Faiss, this is controlled via the `IndexIVF.by_residual` flag, which is set to true by default.

Figure 5 shows that encoding residuals is beneficial for shorter codes. For larger codes, the contribution

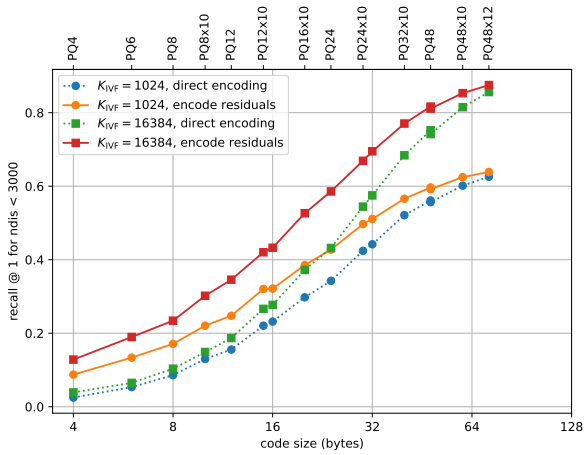


Figure 5: Comparing IVF indexes with and without residual encoding for $K_{\mathrm{IVF}} \in \{2^{10}, 2^{14}\}$ on the Deep1M dataset (d=96 dimensions), with different product quantization settings. We measure the recall that can be achieved within 3000 distance comparisons.

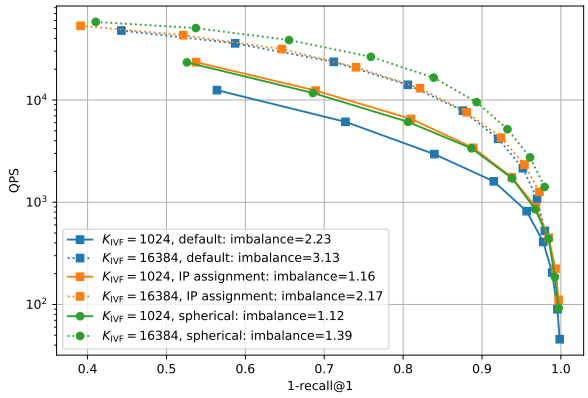


Figure 6: Precision vs. speed trade-off for the MIPS contriever1M dataset. The compared settings are whether the coarse quantizer assignement is done using L2 distance (default) or IP assignment and whether the k-means clustering does a normalization at each iteration (spherical, IP and L2 assignment are equivalent in that case). The imbalance factors are indicated for each setting.

of the residual is less important. Indeed, as the original data is 96-dimensional, it can be compressed to 64 bytes relatively accurately. Note that using higher K_{IVF} also improves the accuracy of the quantizer with residual encoding. From a pure encoding point of view, the additional bits of information brought by the coarse quantizer ($\log_2(K_{\mathrm{IVF}}) = 10$ or 14) improve the accuracy more when used in this residual encoding than if they would added to increase the size of a PQ.

Spherical clustering for inner product search. Efficient indexing for maximum inner product search (MIPS) faces multiple issues: the distribution of query vectors is often different from the database vector distribution, most notably in recommendation systems [65]; the MIPS datasets are diverse, an algorithm that obtains a good performance on some dataset will perform badly on another. Besides, [60] show that using the preprocessing formulas in Section 3 is a suboptimal way of indexing for MIPS.

Several specialized clustering and indexing methods were developed for MIPS [36, 60]. Instead, Faiss implements a modification of k-means clustering, spherical k-means [25], which normalizes the IVF centroids at each iteration. One of the MIPS issues is due to database vectors of very different norms (when they are normalized, MIPS is equivalent to L2 search). High-norm centroids "attract" the database vectors in their clusters, which increases the imbalance factor. Spherical k-means is designed to avoid this issue.

Figure 6 shows that for the Contriever MIPS dataset, the imbalance factor is high. It is reduced by using IP assignment instead of L2, and even more with spherical k-means.

Big batch search. A common use case for ANNS is search with very large query batches. This appears for applications such as large-scale data deduplication. In this case, rather than loading an entire index in memory and processing queries one small batch at a time, it can be more memory-efficient to load only the quantizer, quantize the queries, and then iterate over the index by loading it one chunk at a time. Big-batch search is implemented in the module `contrib.big_batch_search`.

5.2 Graph based

Graph-based indexing consists in building a directed graph whose nodes are the vectors to index. At search time, the graph is explored by following the edges towards the nodes that are closest to the query vector. In practice, the search is not greedy but maintains a priority queue with the most promising edges to explore. Thus, the trade-off at search time is given by the number of exploration steps: higher is more **accurate** but **slower**.

A graph-based algorithm is a general framework that can encompass many variants. In particular, tree-based search or IVF can be seen as special cases of graphs. One can see graphs as a way to precompute neighbors for the database vectors, then match the query to one of the vertices and follow the neighbors from there. However, they can also be built to handle out-of-distribution queries [42, 17].

Given this search algorithm, relying on a pure k-nearest neighbor graph is not optimal because neighbors are redundant. Therefore, the graph building heuristic consists in balancing edges to nearest neighbors and edges that reach more distant nodes. Most graph methods fix the number of outgoing edges per node, which adjusts the trade-off between **search speed** and **memory usage**. The memory usage per vector breaks down into (1) the possibly compressed vector and (2) the outgoing edges for that vector [29].

Faiss implements two graph-based algorithms: HNSW and NSG, respectively in the `IndexHNSW` and `IndexNSG` classes.

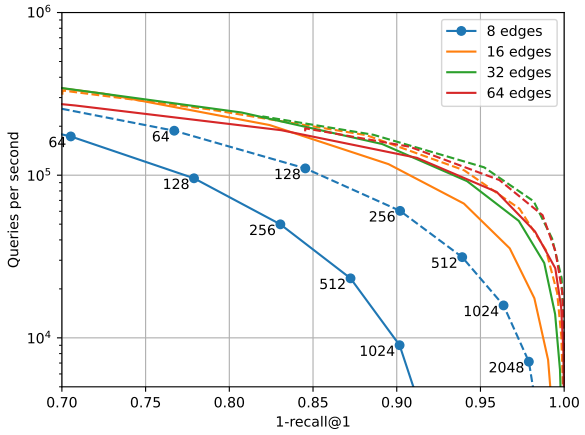


Figure 7: Comparison of graph-based indexing methods HNSW (full lines) and NSG (dashes) to index Deep1M. We sweep the trade-offs between **speed** and **accuracy** by varying the number of graph traversal steps (indicated for some of the curves).

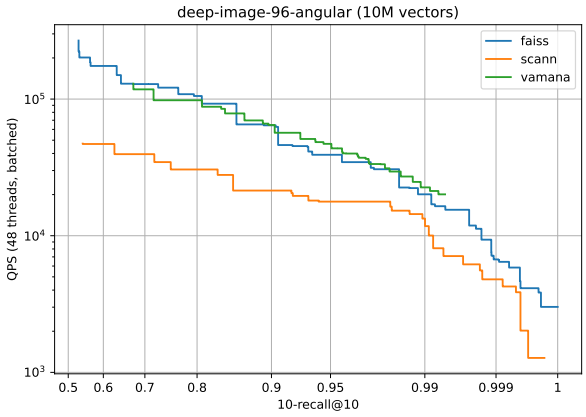


Figure 8: Comparison between the Faiss two other vector search libraries (SCANN and Diskann/vamana) when Deep10M in the ann-benchmarks setup (batch mode).

HNSW. The hierarchical navigable small world graph [55] is a search structure where some randomly selected vertices are promoted to be hubs that are explored first. A notable advantage of HNSW is its ability to add vectors on-the-fly.

NSG. The Navigating Spreading-out Graph [32] is built from a k-nearest neighbor graph that must be provided on input. At building time, some short-range edges are replaced with longer-range edges. The input k-nn graph can be built with a brute force algorithm or with a specialized method such as NN-descent [26] (`NNDescent`). Unlike HNSW, NSG does not rely on multi-layer graph structures, but uses long connections to achieve fast navigation. In addition, NSG starts from a fixed center point when searching.

Discussion. Figure 7 compares the speed-accuracy trade-off for the NSG and HNSW indexes. Their main build-time hyperparameter is the number of edges per node, so we tested several settings (for HNSW this is the number of edges on the base level of the hierarchical graph). The main search-time parameter is the number of graph traversal steps during search (parameter `efSearch` for HNSW and `search_L` for NSG), which we vary to plot each curve. Increasing the number of edges improves the results until 64 edges, beyond which performance deteriorates. NSG obtains better trade-offs in general, at the cost of a longer **build time**. Building the k-NN graph with NN-descent for 1M vectors takes 37 s, and about the same time with exact, brute force search on a GPU. The NSG graph is frozen after the first batch of vectors is added, there is no easy way to add more vectors afterwards.

5.3 How to choose an index

In most cases, choosing an appropriate index can be done by following a process delineated in Appendix A.5. First, one has to decide whether indexing is needed at all: indeed, in some cases a direct brute force search is the best option. Otherwise, the choice is between IVF and graph-based indexes.

IVF vs. graph-based. Graph-based indices are a good option for indexes where there is no constraint on **memory usage**, typically for indexes below 1M vectors. Beyond 10M vectors, the **construction time** typically becomes the limiting factor. For larger indexes, where compression is required to even fit the database vectors in memory, IVF indexes are the only option.

The decision tree of Figure 10 provides intial directions. The Faiss wiki[5] features comprehensive benchmarks for various database sizes and memory budgets. To refine the index parameters, benchmarking should be used.

5.4 Benchmarking indexes

Faiss includes a benchmarking framework (`bench_fw`) that optimizes index types and parameters to explore **accuracy**, **memory usage** and **search time** operating points. The benchmark generates candidate index configurations to evaluate, sweeps both construction-time and search-time parameters, and measures these metrics.

Decoupling encoding and non-exhaustive search options. Beyond a certain scale, **search time** is determined by the number of distance computations performed between the query vector and database vectors.

As shown in Sections 5 and 4, Faiss indexes are built as a combination of pruning and compression, see Table 4. To evaluate index configurations efficiently, the benchmarking framework takes advantage

[5] `https://github.com/facebookresearch/faiss/wiki/Indexing-1G-vectors`

of this compositional design. The training of vector transformations and k-means clustering for IVF coarse quantizers are factored out and reused when possible. Coarse quantizers and IVF indices are first trained and evaluated separately, the parameter space is pruned as described in the previous section, and only the combinations of Pareto-optimal components are benchmarked together.

5.5 Comparison with other libraries

ANN-benchmarks [3] is a codebase that compares several ANNS implementations. We use this setup to compare Faiss with SCANN [36] and Vamana [79], two other industry-standard packages for vector search. We run the search in the following setting: batch search on the Deep10M dataset with 10-recall@10 as the metric, where the training is performed on the database vectors. This differs slightly from the evaluation protocol used originally for this dataset.

Figure 8 shows that Faiss is faster than SCANN and about on-par with Vamana, depending on the operating point. For Faiss we used IVF with a Product Residual Quantizer optimized for SIMD (see Appendix A.3), followed by re-ranking (Section 3.5).

6 Database operations

In the experiments above, the indexes are built in one go with all the vectors, while search operations are performed with one batch containing all query vectors. In real settings, the index evolves over time, vectors may be dynamically added or removed, searches may have to take into account metadata, etc. In this section we show how Faiss supports some of these operations. Specific APIs are available to interface with external storage (Appendix A.4) if fine-grained control is required.

6.1 Identifier-based operations

Faiss indexes support two types of identifiers: sequential and arbitrary ids. Sequential ids are based on the order of additions in the index. Alternatively, the user can provide arbitrary 63-bit integer ids associated to each vector (the sign bit is reserved for invalid results). The corresponding addition methods for the index are `add` and `add_with_ids`. Unlike e.g. Usearch [87], Faiss does not store arbitrary metadata with the vectors.

Index updates. The Faiss API includes methods to remove vectors (`remove_ids`) and update them (`update_vectors`) by passing the corresponding ids.

Note that if the vector *distribution* changes significantly because of additions/removals or updates, then the efficiency of any technique that fits the data distribution degrades. This includes IVF and PQ compression. This degradation can be addressed by explicit updates of the index structure [8].

Flat indexes. Sequential indexes (`IndexFlatCodes`) store vectors as an array. They support only sequential ids. When arbitrary ids are needed, the index can be embedded in a `IndexIDMap`, that translates sequence numbers to arbitrary ids using a `int64` array. This enables `add_with_ids` and returns the arbitrary ids at search time. `IndexIDMap2` in addition maps arbitrary ids back to sequential ids using a hash table. The graph indexes rely on an embedded `IndexFlatCodes` to store the actual vectors. To use them with non-sequential ids, they should be wrapped with the same mapping objects.

IVF indexes. The IVF indexing structure supports user-provided ids natively at addition and search time. However, id-based access may require a sequential scan, as the entries are stored in an arbitrary order in the inverted lists. Therefore, the IVF index can optionally maintain a `DirectMap`, which maps user-visible ids to the inverted list and the offset they are stored in. It supports lookup, removal and update by ids. The map can be an array, which is appropriate for sequential ids, or a hash table, for arbitrary 63-bit ids. The direct map incurs a **memory overhead** and an **add-time computation overhead**, therefore, it is disabled by default.

Graph indexes. The graph index HNSW does not support suppression and mutation, and NSG does not even support adding vectors incrementally. Supporting this requires heuristics to rebuild the graph when it is mutated, which are implemented in HNSWlib and FreshDiskANN [78] but suboptimal indexing-wise.

6.2 Filtered search

Vector filtering consists in returning only database vectors based on some search-time criterion, other vectors are ignored. Faiss has basic support for vector filtering: the user can provide a predicate (`IDSelector` callback), and if the predicate returns false on the vector id, the vector is ignored.

Therefore, if metadata is needed to filter the vectors, the callback function needs to do an indirection to the metadata table, which is inefficient. Another approach is to exploit the unused bits of the identifier. If N documents are indexed with sequential ids, $63 - \lceil \log_2(N) \rceil$ bits are unused.

This is sufficient to store enumerated types (e.g. country codes, music genres, license types, etc.), dates (as days since some origin), version numbers, etc. However, it is insufficient for more complex metadata. In the example use case below, we use the available bits to implement more complex filtering.

Filtering with bag-of-word vectors. In the filtered search track of the BigANN 2023 competition [75], each query and database vector is associated with a few terms from a fixed vocabulary of size v (for the

queries there are only 1 or 2 words). The filtering consists in considering only the database vectors that include all the query terms. . This metadata is given as a sparse matrix $M_{\text{meta}} \in \{0,1\}^{N \times v}$.

The basic implementation of the filter starts from query vector q and the associated words $w_1, w_2 \in \{1...v\}$. Before computing a distance to a vector with id i, it fetches row i of M_{meta} to verify that w_1 and w_2 are in it. This predicate is slow because (1) it requires to access M_{meta}, which causes cache misses and (2) it performs an iterative binary search in the sparse matrix structure. Since the callback is called in the tightest inner loop of the search function, and since the IVF search tends to perform many vector comparisons, this has non negligible performance impact.

To speed up the predicate, we can use bit manipulations. In this example, $N = 10^7$, so we use only $\lceil \log_2 N \rceil = 24$ bits of the ids, leaving $63 - 24 = 39$ bits that are always 0. We associate to each word j a 39-bit signature $S[j]$, and to each set of words the binary "or" of these signatures. The query is represented by $s_{\text{q}} = S[w_1] \vee S[w_2]$. Database entry i with words W_i is represented by $s_i = \vee_{w \in W_i} S[w]$. Then the following implication holds: if $\{w_1, w_2\} \subset W_i$ then all 1 bits of s_{q} are also set to 1 in s_i:

$$\{w_1, w_2\} \subset W_i \Rightarrow \neg s_i \wedge s_{\text{q}} = 0, \tag{19}$$

which is equivalent to:

$$\neg s_i \wedge s_{\text{q}} \neq 0 \Rightarrow \{w_1, w_2\} \not\subset W_i. \tag{20}$$

This binary test costs only a few machine instructions on data that is already in machine registers. It can thus be used as a pre-filter before applying the predicate computation. This is implemented in the module `bow_id_selector`[6].

The remaining degree of freedom is how to choose the binary signatures, because this rule's filtering ability depends on the choice of the signatures S. We experimented with i.i.d. Bernoulli bits with varying p: the best setting avoids running the full predicate more than 4/5 times.

p = probability of 1	0.05	0.1	0.2	0.5
Filter hit rate	75.4%	82.1%	76.4%	42.0%

Vector-first or metadata-first search. There are two possible approaches to filtered search: *vector-first*, which is described above, and *metadata-first*, where only vectors with appropriate metadata are considered in vector search. The metadata-first filtering generates a subset of vectors to compare with that can then be compared using brute force search. Brute force search is slow but acceptable if the subset size is small, and the results are exact.

Therefore, in the context of the BigANN competition [75], the decision to use vector-first or metadata-first depends on how large the subset is. To this end, we map each word w to the list of items that contain w, of size L_w.

If there is a single query word $\{w_1\}$ then the subset size is directly accessible as $S = L_{w_1}$. With two query words $\{w_1, w_2\}$, finding the subset size requires intersecting the inverted lists for w_1 and w_2, which is slow. Instead, one can estimate the size of the subset using the empirical probability of each word to appear $P(w) = L_w/N$. Assuming independent draws, the probability of both words to appear is $P(w_1)P(w_2)$. Therefore, the expected size of the intersection is $S \approx NP(w_1)P(w_2) = L_{w_1}L_{w_2}/N$. If $S/N < 3 \times 10^{-4}$ (an empirical threshold), then the subset is sufficiently small that metadata-first can be applied. Of course, if metadata-first is selected, the actual intersection has to be computed. Most participants to the competition used similar heuristics.

7 Faiss applications

Faiss is widely used across the industry, with numerous applications leveraging its capabilities. The following examples highlight notable use cases that demonstrate exceptional scalability or significant impact.

7.1 Trillion scale index

In this example, we index 1.5 trillion vectors in 144 dimensions. The indexing needs to be accurate, therefore the compression of the vectors is limited to 54 bytes with a PCA to 72 dimensions and 6-bit scalar quantizer (`PCAR72,SQ6`).

A HNSW coarse quantizer with 10M centroids is used for the `IndexIVFScalarQuantizer`, trained with a simple distributed GPU k-means (implemented in `faiss.clustering`).

Once the training is completed, the index is built in the following three phases:

1. shard over ids: add the input vectors in 2000 shards independently, producing 2000 indexes (each one fits in 256 GB RAM);
2. shard over lists: build the 100 indexes corresponding each to a subset of 100k inverted lists. This is done on 100 different machines, each reading from the 2000 sharded indices, and writing the results directly to a distributed file system;
3. load the shards: memory-map all 100 indexes on a central machine as 100 `OnDiskInvertedLists` (a memory map of 83 TiB).

Steps 1 and 2 are organized to be performed as independent cluster jobs on a few hundred servers (64 cores, 256G RAM). Otherwise, the code is written in standard Faiss in Python.

The central machine that handles searches performs the coarse quantization and loads the inverted lists from the distributed disk partition. The limiting factor is the network bandwidth of this central machine.

[6] https://github.com/harsha-simhadri/big-ann-benchmarks/tree/main/neurips23/filter/faiss

Therefore, it is more efficient to distribute the search on 20 intermediate servers to spread the load. This brings the search time down to roughly 1 s per query.

7.2 Text retrieval

Faiss is commonly used for knowledge intensive natural language processing tasks. In particular, ANNS is relevant for information retrieval [85, 67], with applications such as fact checking, entity linking, slot filling or open-domain question answering: these often rely on retrieving relevant content across a large-scale corpus. To that end, embedding models have been optimized for text retrieval [40, 49].

Finally, [41], [50], [74] and [48] consist of language models that have been trained to integrate textual retrieval in order to improve their accuracy, factuality or compute efficiency.

7.3 Data mining

Another recurrent application of ANNS and Faiss is in the mining and curation of large datasets. In particular, Faiss has been used to mine bilingual texts across very large text datasets retrieved from the web [72, 9], or to organize a language model's training corpus in order to group together series of documents covering similar topics [73].

In the image domain, [64] leverages Faiss to remove duplicates from a dataset containing 1.3B images. It then relies on efficient indexing in order to mine a curated dataset whose distribution matches the distribution of a target dataset.

7.4 Content Moderation

One of the major applications of Faiss is the detection and remediation of harmful content at scale. Human-labeled examples of policies violating images and videos are embedded with models such as SSCD [69] and stored in a Faiss index. To decide if a new image or video would violate some policies, a multi-stage classification pipeline first embeds the content and searches the Faiss index for similar labeled examples, typically utilizing range queries. The results are aggregated and processed through additional machine classification or human verification. Since the impact of mistakes is high, good representations should discriminate perceptually similar and different content, and accurate similarity search is required even at billion to trillion scale. The former problem motivated the Image and Video Similarity Challenges [30, 68].

8 Conclusion

Throughout the years, Faiss continuously expanded its focus to include the most relevant vector indexing techniques from research. We continue doing this to include novel quantization techniques [39], better hardware support for some indexes [63] and new indexing forms, such as associative vector memories for transformer architectures [20, 94].

References

[1] Kenza Amara, Matthijs Douze, Alexandre Sablayrolles, and Hervé Jégou. Nearest neighbor search with compact codes: A decoder perspective. In *ICMR*, 2022.

[2] Alexandr Andoni and Ilya Razenshteyn. Optimal data-dependent hashing for approximate near neighbors. In *Proceedings of the forty-seventh annual ACM symposium on Theory of computing*, pages 793–801, 2015.

[3] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020.

[4] Artem Babenko and Victor Lempitsky. Additive quantization for extreme vector compression. In *Conference on Computer Vision and Pattern Recognition*, 2014.

[5] Artem Babenko and Victor Lempitsky. Tree quantization for large-scale similarity search and classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4240–4248, 2015.

[6] Artem Babenko and Victor Lempitsky. Efficient indexing of billion-scale datasets of deep descriptors. In *Conference on Computer Vision and Pattern Recognition*, 2016.

[7] Yoram Bachrach, Yehuda Finkelstein, Ran Gilad-Bachrach, Liran Katzir, Noam Koenigstein, Nir Nice, and Ulrich Paquet. Speeding up the xbox recommender system using a euclidean transformation for inner-product spaces. In *Proceedings of the 8th ACM Conference on Recommender systems*, pages 257–264, 2014.

[8] Dmitry Baranchuk, Matthijs Douze, Yash Upadhyay, and I Zeki Yalniz. Dedrift: Robust similarity search under content drift. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11026–11035, 2023.

[9] Loïc Barrault, Yu-An Chung, Mariano Cora Meglioli, David Dale, Ning Dong, Paul-Ambroise Duquenne, Hady Elsahar, Hongyu Gong, Kevin Heffernan, John Hoffman, et al. Seamlessm4t-massively multilingual & multimodal machine translation. *arXiv preprint arXiv:2308.11596*, 2023.

[10] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the*

association for computational linguistics, 5:135–146, 2017.

[11] Leonid Boytsov, David Novak, Yury Malkov, and Eric Nyberg. Off the beaten path: Let's replace term-based retrieval with k-nn search. In *Proceedings of the 25th ACM international on conference on information and knowledge management*, pages 1099–1108, 2016.

[12] Sebastian Bruch, Franco Maria Nardini, Amir Ingber, and Edo Liberty. An approximate algorithm for maximum inner product search over streaming sparse vectors. *arXiv preprint arXiv:2301.10622*, 2023.

[13] Zhangjie Cao, Mingsheng Long, Jianmin Wang, and Philip S. Yu. Hashnet: Deep learning to hash by continuation. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[14] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze. Deep clustering for unsupervised learning of visual features. In *Proceedings of the European conference on computer vision (ECCV)*, pages 132–149, 2018.

[15] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9650–9660, 2021.

[16] Moses Charikar. Similarity estimation techniques from rounding algorithms. In *Proc. ACM symp. Theory of computing*, 2002.

[17] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X Sean Wang. Roargraph: A projected bipartite graph for efficient cross-modal approximate nearest neighbor search. *arXiv preprint arXiv:2408.08933*, 2024.

[18] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, Proceedings of Machine Learning Research, pages 1597–1607. PMLR, 2020.

[19] Yongjian Chen, Tao Guan, and Cheng Wang. Approximate nearest neighbor search by residual vector quantization. *Sensors*, 10(12):11259–11273, 2010.

[20] Zhuoming Chen, Ranajoy Sadhukhan, Zihao Ye, Yang Zhou, Jianyu Zhang, Niklas Nolte, Yuandong Tian, Matthijs Douze, Leon Bottou, Zhihao Jia, et al. Magicpig: Lsh sampling for efficient llm generation. *arXiv preprint arXiv:2410.16179*, 2024.

[21] Felix Chern, Blake Hechtman, Andy Davis, Ruiqi Guo, David Majnemer, and Sanjiv Kumar. Tpu-knn: K nearest neighbor search at peak flop/s. *Advances in Neural Information Processing Systems*, 35:15489–15501, 2022.

[22] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*, pages 253–262, 2004.

[23] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[25] Inderjit S Dhillon and Dharmendra S Modha. Concept decompositions for large sparse text data using clustering. *Machine learning*, 42:143–175, 2001.

[26] Wei Dong, Charikar Moses, and Kai Li. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th international conference on World wide web*, pages 577–586, 2011.

[27] Matthijs Douze and Hervé Jégou. The yael library. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 687–690, 2014.

[28] Matthijs Douze, Hervé Jégou, and Florent Perronnin. Polysemous codes. In *European Conference on Computer Vision*, 2016.

[29] Matthijs Douze, Alexandre Sablayrolles, and Hervé Jégou. Link and code: Fast indexing with graphs and compact regression codes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3646–3654, 2018.

[30] Matthijs Douze, Giorgos Tolias, Ed Pizzi, Zoë Papakipos, Lowik Chanussot, Filip Radenovic, Tomas Jenicek, Maxim Maximov, Laura Leal-Taixé, Ismail Elezi, et al. The 2021 image similarity dataset and challenge. *arXiv preprint arXiv:2106.09672*, 2021.

[31] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. Sentence-level multimodal and language-agnostic representations. *arXiv preprint arXiv:2308.11466*, 2023.

[32] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *arXiv preprint arXiv:1707.00143*, 2017.

[33] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization for approximate nearest neighbor search. In *Conference on Computer Vision and Pattern Recognition*, 2013.

[34] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*, 2023.

[35] Yunchao Gong, Svetlana Lazebnik, Albert Gordo, and Florent Perronnin. Iterative quantization: A procrustean approach to learning binary codes for large-scale image retrieval. IEEE *Trans. Pattern Analysis and Machine Intelligence*, 2012.

[36] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*. PMLR, 2020.

[37] Weixiang Hong, Xueyan Tang, Jingjing Meng, and Junsong Yuan. Asymmetric mapping quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(7):1783–1790, 2019.

[38] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. Embedding-based retrieval in facebook search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2553–2561, 2020.

[39] Iris A.M. Huijben, Matthijs Douze, Matthew J. Muckley, Ruud J.G. van Sloun, and Jakob Verbeek. Residual quantization with implicit neural codebooks. In *International Conference on Machine Learning (ICML)*, 2024.

[40] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning, 2021.

[41] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.

[42] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. Ood-diskann: Efficient and scalable graph anns for out-of-distribution queries, 2022.

[43] Herve Jegou, Matthijs Douze, and Cordelia Schmid. Hamming embedding and weak geometric consistency for large scale image search. In *Computer Vision–ECCV 2008: 10th European Conference on Computer Vision, Marseille, France, October 12-18, 2008, Proceedings, Part I 10*, pages 304–317. Springer, 2008.

[44] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. IEEE *Trans. Pattern Analysis and Machine Intelligence*, 2010.

[45] Hervé Jégou, Florent Perronnin, Matthijs Douze, Jorge Sánchez, Patrick Pérez, and Cordelia Schmid. Aggregating local image descriptors into compact codes. *IEEE transactions on pattern analysis and machine intelligence*, 34(9):1704–1716, 2011.

[46] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. Searching in one billion vectors: re-rank with source coding. In *International Conference on Acoustics, Speech, and Signal Processing*, 2011.

[47] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Trans. on Big Data*, 2019.

[48] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models, 2020.

[49] Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen tau Yih, and Xilun Chen. How to train your dragon: Diverse augmentation towards generalizable dense retrieval, 2023.

[50] Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Scott Yih. Radit: Retrieval-augmented dual instruction tuning. *ArXiv*, abs/2310.01352, 2023.

[51] Shicong Liu, Hongtao Lu, and Junru Shao. Improved residual vector quantization for high-dimensional approximate nearest neighbor search. *arXiv preprint arXiv:1509.05195*, 2015.

[52] Stuart Lloyd. Least squares quantization in PCM. IEEE *Transactions on Information Theory*, 1982.

[53] David G Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2), 2004.

[54] Qin Lv, Moses Charikar, and Kai Li. Image similarity search with compact data structures. In *Proceedings of the thirteenth ACM international conference on Information and knowledge management*, pages 208–217, 2004.

[55] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[56] Julieta Martinez, Joris Clement, Holger H Hoos, and James J Little. Revisiting additive quantization. In *European Conference on Computer Vision*, 2016.

[57] Julieta Martinez, Shobhit Zakhmi, Holger H Hoos, and James J Little. LSQ++: lower running time and higher recall in multi-codebook quantization. In *European Conference on Computer Vision*, 2018.

[58] Yusuke Matsui, Yusuke Uchida, Hervé Jégou, and Shin'ichi Satoh. A survey of product quantization. *ITE Transactions on Media Technology and Applications*, 2018.

[59] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[60] Stanislav Morozov and Artem Babenko. Non-metric similarity graphs for maximum inner product search. *Advances in Neural Information Processing Systems*, 31, 2018.

[61] Stanislav Morozov and Artem Babenko. Unsupervised neural quantization for compressed-domain similarity search. In *International Conference on Computer Vision*, 8 2019.

[62] Marius Muja and David G Lowe. Scalable nearest neighbor algorithms for high dimensional data. *IEEE transactions on pattern analysis and machine intelligence*, 36(11):2227–2240, 2014.

[63] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus, 2023.

[64] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision, 2023.

[65] Arkadiusz Paterek. Improving regularized singular value decomposition for collaborative filtering. In *Proceedings of KDD cup and workshop*, 2007.

[66] Loïc Paulevé, Hervé Jégou, and Laurent Amsaleg. Locality sensitive hashing: A comparison of hash function types and querying mechanisms. *Pattern recognition letters*, 31(11):1348–1358, 2010.

[67] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. KILT: a benchmark for knowledge intensive language tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Online, June 2021. Association for Computational Linguistics.

[68] Ed Pizzi, Giorgos Kordopatis-Zilos, Hiral Patel, Gheorghe Postelnicu, Sugosh Nagavara Ravindra, Akshay Gupta, Symeon Papadopoulos, Giorgos Tolias, and Matthijs Douze. The 2023 video similarity dataset and challenge. *arXiv preprint arXiv:2306.09489*, 2023.

[69] Ed Pizzi, Sreya Dutta Roy, Sugosh Nagavara Ravindra, Priya Goyal, and Matthijs Douze. A self-supervised descriptor for image copy detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14532–14542, 2022.

[70] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

[71] Harsimrat Sandhawalia and Hervé Jégou. Searching with expectations. In *International Conference on Acoustics, Speech, and Signal Processing*, 2010.

[72] Holger Schwenk and Matthijs Douze. Learning joint multilingual sentence representations with neural machine translation. In *Proceedings of the 2nd Workshop on Representation Learning for NLP*, pages 157–167, Vancouver, Canada, August 2017. Association for Computational Linguistics.

[73] Weijia Shi, Sewon Min, Maria Lomeli, Chunting Zhou, Margaret Li, Victoria Lin, Noah A. Smith, Luke Zettlemoyer, Scott Yih, and Mike Lewis. In-context pretraining: Language modeling beyond document boundaries. *ArXiv*, abs/2310.10638, 2023.

[74] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen tau Yih. Replug: Retrieval-augmented black-box language models, 2023.

[75] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, Dmitry Baranchuk, Edo Liberty, Frank Liu, Ben Landrum, et al. Results of the big ann: Neurips'23 competition. *arXiv preprint arXiv:2409.17424*, 2024.

[76] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamny, Gopal Srinivasa, et al. Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search. In *NeurIPS 2021 Competitions and Demonstrations Track*, pages 177–189. PMLR, 2022.

[77] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamny, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. Results of the neurips'21 challenge on billion-scale approximate nearest neighbor search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, volume 176 of *Proceedings of Machine Learning Research*, pages 177–189. PMLR, 06–14 Dec 2022.

[78] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search, 2021.

[79] Suhas Jayaram Subramanya, Rohan Kadekodi, Ravishankar Krishaswamy, and Harsha Vardhan Simhadri. Diskann: Fast accurate billion-point nearest neighbor search on a single node. In *Neurips*, 2019.

[80] Philip Sun, Ruiqi Guo, and Sanjiv Kumar. Automating nearest neighbor search configuration with constrained optimization. *arXiv preprint arXiv:2301.01702*, 2023.

[81] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. Soar: Improved quantization for approximate nearest neighbor search. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[82] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1–9, 2015.

[83] Gergely Szilvasy, Pierre-Emmanuel Mazaré, and Matthijs Douze. Vector search with small radiuses, 2024.

[84] Romain Tavenard, Hervé Jégou, and Laurent Amsaleg. Balancing clusters to reduce response time variability in large scale image search. In *2011 9th International Workshop on Content-Based Multimedia Indexing (CBMI)*, pages 19–24. IEEE, 2011.

[85] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021.

[86] Bob van Luijt and Micha Verhagen. Bringing semantic knowledge graph technology to your data. *IEEE Software*, 37(2):89–94, 2020.

[87] Ash Vardanian. USearch by Unum Cloud, June 2022.

[88] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[89] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*, pages 2614–2627, 2021.

[90] Jingdong Wang, Ting Zhang, Nicu Sebe, Heng Tao Shen, et al. A survey on learning to hash. *IEEE transactions on pattern analysis and machine intelligence*, 40(4):769–790, 2017.

[91] Jun Wang, Wei Liu, Sanjiv Kumar, and Shih-Fu Chang. Learning to hash for indexing big data - a survey. *Proc. of the IEEE*, 2015.

[92] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *arXiv preprint arXiv:2101.12631*, 2021.

[93] Roger Weber, Hans-Jörg Schek, and Stephen Blott. A quantitative analysis and performance

study for similarity-search methods in high-dimensional spaces. In *VLDB*, volume 98, pages 194–205, 1998.

[94] Jianyu Zhang, Niklas Nolte, Ranajoy Sadhukhan, Beidi Chen, and Léon Bottou. Memory mosaics, 2024.

A Appendix

This appendix exposes aspects of Faiss's implementation. Faiss started in a research environment. As a consequence, it grew organically as indexing research was making progress.

In the following, we summarize the guiding principles that keep the library coherent (Appendix A.1); the structure of the library and its dependencies (Appendix A.2); how optimization is performed (Appendix A.3); an example of how Faiss internals are exposed so that it can be embedded in a vector database (Appendix A.4; and finally, a flowchart that shows how to choose a Faiss index (Appendix A.5).

A.1 Code structure

The core of Faiss is implemented in C++. The guiding principles are (1) the code should be as open as possible, so that users can access all the implementation details of the indexes; (2) Faiss should be easy to embed from external libraries; (3) the core library focuses on vector search only.

Therefore, all fields of the classes are public (C++ `struct`). Faiss is a late adopter for C++ standards, so that it can be used with relatively old compilers (currently C++17).

Faiss's basic data types are concrete (not templates): vectors are always represented as 32-bit floats that are portable and provide a good trade-off between **size** and **accuracy**. Similarly, all vector ids are represented with 64-bit integers. This is often larger than necessary for sequential numbering but is widely used for database identifiers.

Faiss is modular and includes few dependencies, so that linking it from C++ is easy. Callback classes (`ResultHanlder`, `InvertedLists`, `IDSelector`) can be subclassed to customize the indices. On the Python level, they can be wrapped in SWIG to be provided to Faiss without recompiling Faiss itself.

A.2 High-level interface

Figure 9 shows the structure of the library. The C++ core library and the GPU add-on have as few dependencies as possible: only a BLAS implementation and CUDA itself.

In order to facilitate experimentation, the whole library is wrapped for Python with numpy. To this end, SWIG[7] exhaustively generates wrappers for all C++ classes, methods and variables. The associated Python layer also contains benchmarking code, dataset definitions, driver code. More and more functionality is embedded in the `contrib` package of Faiss. Faiss also provides a pure C API, which is useful for bindings with programming languages such as Rust or Java.

The `Index` is presented to the end user as a monolithic object, even when it embeds other indexes

[7] https://www.swig.org/

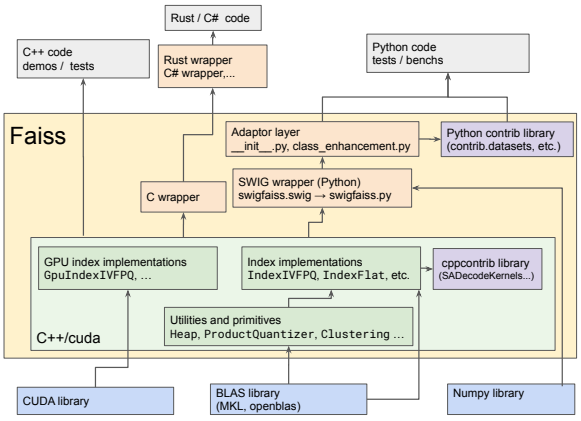


Figure 9: Architecture of the Faiss library. Arrows indicate dependencies. *Bottom:* the library's dependencies. *Top:* example of software that depends on Faiss, most notably its extensive test suite.

as quantizers, refinement indexes or sharded sub-indexes. Therefore, an index can be duplicated with `clone_index` and serialized into as a single byte stream using a single function, `write_index`. It also contains the necessary headers so that it can be read by a generic function, `read_index`.

The index factory. Index objects can be instantiated explicitly in C++ or Python, but it is more common to build them with the `index_factory` function. This function takes a string that describes the index structure and its main parameters. For example, the string `PCA160,IVF20000_HNSW,PQ20x10,RFlat` instantiates an IVF index with $K_{\mathrm{IVF}} = 20000$, where the coarse quantizer is a HNSW index; then the vectors are represented with a `PQ20x10` product quantizer. The data is preprocessed with a PCA to 160 dimensions, and the search results are re-ranked with a refinement index that performs exact distance computations. All the index parameters are set to reasonable defaults, e.g. the PQ encodes the residual of the vectors w.r.t. the coarse quantization centroids. Faiss indexes can be used as vector codecs with functions `sa_encode`, `sa_decode` and `sa_code_size`.

A.3 Optimization

Approach to optimization. Faiss aims at being feature complete first. A non-optimal version of all indexes is implemented first. Code is optimized only when it appears that **runtime** is important for a certain index. The non-optimized setting is used to control the correctness of the optimized version.

Often, only a subset of data sizes are optimized. For example, for PQ indexes, only $K = 2^8$ and $K = 2^4$ and $d/M \in \{2, 4, 8, 16, 20\}$ are fully optimized. For `IndexLSH` search, we only optimized code sizes 4, 8, 16 and 20. Fixing these sizes allows to write dedicated "kernels", *i.e.*, sequences of instructions without explicit loops or tests, that aim to maximize arithmetic throughput.

When generic scalar CPU optimizations are exhausted, Faiss also optimizes specifically for some hardware platforms.

CPU vectorization. Modern CPUs support Single Instruction, Multiple Data (SIMD) operations, specifically AVX/AVX2/AVX512 for x86 and NEON for ARM. Faiss exploits those at three levels.

When operations are simple enough (e.g. elementwise vector sum), the code is written in a way that the compiler can vectorize the code by itself, which often boils down to adding `restrict` keywords to promise that arrays are not overlapping.

The second level leverages SIMD variables and instructions through C++ compiler extensions. Faiss includes `simdlib`, a collection of classes intended as a layer above the AVX and NEON instruction sets. However, much of the SIMD is done specifically for one instruction set – most often AVX – because it is more efficient.

The third level of optimization adapts the data layout and algorithms in order to speed up their SIMD implementation. The 4-bit product and additive quantizer implementations are implemented in this way, inspired by the SCANN library [36]: the layout of the PQ codes for several consecutive vectors is interleaved in memory so that a vector permutation can be used to perform the LUT lookups of (14) in parallel. This is implemented in the `FastScan` variants of PQ and AQ indexes (`IndexPQFastScan`, `IndexIVFResidual QuantizerFastScan`, etc.).

GPU Faiss. Porting Faiss to the GPU is an involved undertaking due to substantial architectural specificities. The implementation of GPU Faiss is detailed in [47], we summarize the GPU implementation challenges therein.

Modern multi-core CPUs are highly latency optimized: they employ an extensive cache hierarchy, branch prediction, speculative execution and out-of-order code execution to improve serial program execution. In contrast, GPUs have a limited cache hierarchy and omit many of these latency optimizations. They instead possess a larger number of concurrent threads of execution (Nvidia's A100 GPU allows for up to 6,912 *warps*, each roughly equivalent to a 32-wide vector SIMD CPU thread of execution), a large number of floating-point and integer arithmetic functional units (A100 has up to 19.5 teraflops per second of fp32 fused-multiply add throughput), and a massive register set to allow for a high number of long latency pending instructions in flight (A100 has 27 MiB of register memory). They are thus largely throughput-optimized machines.

The algorithmic techniques used in vector search can be grouped into three broad categories: distance computation of floating-point or binary vectors (which may have been produced via dequantization from a compressed form), table lookups (as seen in PQ distance computations) or scanning (as seen when

traversing IVF lists), and irregular, sequential computations such as linked-list traversal (as used in graph-based indices) or ranking the k closest vectors.

Distance computation is easy on GPUs and readily exceeds CPU performance, as GPUs are optimized for matrix-matrix multiplication such as that seen in `IndexFlat` or `IVFFlat`. Table lookups and list scanning can also be made performant on GPUs, as it is possible to stage small tables (as seen in product quantization) in *shared memory* (roughly a user-controlled L1 cache) or register memory and perform lookups in parallel across all warps.

Sequential table scanning in IVF indices requires loading data from main (*global*) memory. While main memory access latency is high, for table scanning we know in advance what data we wish to access. The data movement from main memory into registers can thus be pipelined or use double buffering, so we can achieve close to peak possible performance.

Selecting the k closest vectors to a query vector by ranking distances on the CPU is best implemented with a min- or max-heap. On the GPU, the sequential operations involved in heap operations would similarly force the GPU into a latency-bound regime. This is the largest challenge for GPU implementation of vector search, as the time needed for the heap implementation is an order of magnitude greater than all other arithmetic. To handle this, we developed an efficient GPU k-selection algorithm [47] that allows for ranking candidate vectors in a single pass, operating at a substantial fraction of peak possible performance per memory bandwidth limits. It relies upon heavy usage of the high-speed, large register memory on GPUs, and small-set bitonic sorting via *warp shuffles* with buffering techniques.

Irregular computations such as walking graph structures for graph-based indices like HNSW tend to remain in the latency-bound (due to the sequential traversal) rather than arithmetic throughput or memory bandwidth-bound regimes. Here, GPUs are at a disadvantage as compared to CPUs, and emerging techniques such as CAGRA [63] are required to parallelize otherwise sequential operations with graph traversal.

GPU Faiss implements brute-force `GpuIndexFlat` as well as the IVF indices `GpuIndexIVFFlat`, `GpuIndexIVF ScalarQuantizer` and `GpuIndexIVFPQ`, which are the most useful for large-scale indexing. The coarse quantizer for the IVF indices can be on either CPU or GPU. The GPU index objects have the same interface as their CPU counterparts and the functions `index_cpu_to_gpu` / `index_gpu_to_cpu` convert between them. Multiple GPUs are also supported. GPU indexes can take inputs and outputs in GPU or CPU memory as input and output, and Python interface can handle Pytorch tensors.

Advanced options for Faiss components and indices. Many Faiss components expose internal parameters to fine-tune the trade-off between metrics: number of iterations of k-means, batch sizes for brute-force distance computations, etc. Default parameter values are set to work reasonably well in most cases.

Multi-threading. Faiss relies on OpenMP to handle multi-threading. By default, Faiss switches to multi-threading processing if it is beneficial, for example, at training and batch addition time. Faiss multi-threading behavior may be controlled with standard OpenMP environment variables and functions, such as `omp_set_num_threads`.

When searching a single vector, Faiss does not spawn multiple threads. However, when batched queries are provided, Faiss processes them in parallel, exploiting the effectiveness of the CPU cache and batched linear algebra operations. This is faster than calling `search` from multiple threads. Therefore, queries should be submitted by batches if possible.

A.4 Interfacing with external storage

Faiss indexes are based on simple storage structures, mainly `std::vector` to make copy-construction easier. The default implementation of `IndexIVF` is based on this storage. However, to give vector database developers more control over the storage of inverted lists, Faiss provides two lower-level APIs.

Arbitrary inverted lists. The IVF index uses an abstract `InvertedLists` object as its storage. The object exposes routines to read one inverted list, add entries to it and remove entries. The default `ArrayInvertedLists` uses in-memory storage. Alternatively, `OnDiskInverted Lists` provides memory-mapped storage.

More complex implementations can access a key-value storage either by storing the entire inverted list as a value, or by utilizing key prefix scan operations like the one supported by RocksDB to treat multiple keys prefixed by the same identifier as one inverted list. To this end, the `InvertedLists` implementation exposes an `InvertedListsIterator` and fetches the codes and ids from the underlying key-value store, which usually exposes a similar iterable interface. Adding, updating and removing codes can be delegated to the underlying key-value store. We provide an implementation for RocksDB in `rocksdb_ivf`.

Scanner objects. With the abstraction above, the scanning loop is still controlled by Faiss. If the calling code needs to control the looping code, then the Faiss IVF index provides an `InvertedListScanner` object. The scanner's state includes the current query vector and current inverted list. It provides a `distance_to_code` method that, given a code, computes the distance from the query to the decompressed vector. At a slightly higher level, it loops over a set of codes and updates a provided result buffer.

This abstraction is useful when the inverted lists are not stored sequentially or fragmented into sub-lists

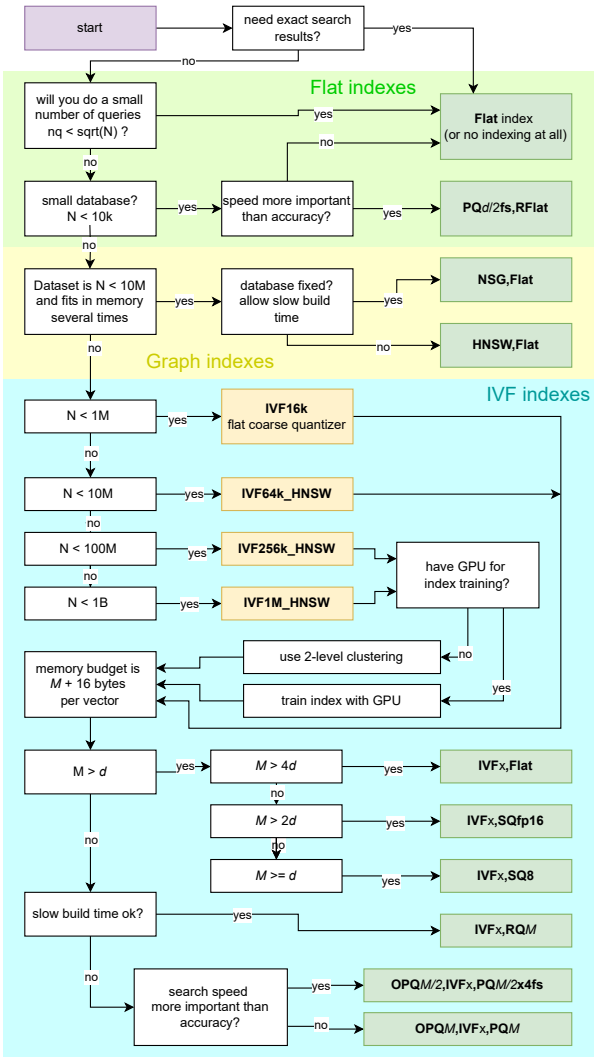


Figure 10: Decision tree to choose a Faiss index. This is for the common case of Euclidean k-nearest neighbor search on CPU. The resulting indexes (in bold) are defined by their factory string (see Appendix A.2).

because of metadata filtering [38]. Faiss is used only to perform the coarse quantization and the vector encoding.

A.5 How to choose an index

Figure 10 shows a decision tree for index types, depending on database size and memory constraints. The decision tree first focuses on the "hard" **memory** constraint, then on secondary trade-offs, like **index construction time** vs. **accuracy**.

A.6 Faiss metric types

Table 3 lists the metric types supported in Faiss. They are inspired by the distance metrics in the Scipy library [88]. Only the two first metric types are supported for all index types. The others are supported only for brute-force computations and non-compressed versions of IVF and HNSW (`IVFFlat` and `HNSWFlat`).

A.7 API index of the Faiss library

As mentioned in Section 5.3, most indexes are a combination of a compression method (like PQ) and a non-exhaustive search method. The index class names are built as shown in Table 4.

Figure 11 shows the most important index classes in Faiss, grouped by broad families. Some combinations are omitted for brevity.

Faiss index API. In the following, we list the main methods offered by Faiss indexes, $x \in \mathbb{R}^{n \times d}$ is a list of n vectors in dimension d represented as the rows of a matrix, and $I \in \{0, 2^{63}-1\}^n$ is a list of 63-bit ids of size n.

- `train(`x`)` perform a training using vectors x to prepare adding vectors of the same data distribution to the index;
- `add(`x`)` add the vectors x to the index, numbered sequentially;
- `add_with_ids(`x`,`I`)` add the vectors x, identified by the 63-bit ids I;
- `search(`x`, `k`)` return the k nearest vectors of each of the query vectors in x;
- `range_search(`x`, `ε`)` return all vectors within a radius ε of each of the query vectors in x;
- `remove_ids(`I`)` remove vectors with ids I from the index;
- `reconstruct_batch(`I`)` extract the vectors with ids in I. It is called "reconstruct" because for most index types, the returned vectors will be approximations of the original ones.

Quantizer objects. Table 5 shows the hierarchy of quantizers. Each quantizer can represent the reproduction values of all the quantizers below, but is slower to train and to perform assignment with.

The root `Quantizer` class has the following fields and methods (where $x \in \mathbb{R}^{n \times d}$ is a list of vectors):

- `code_size` size of the codes it produces (in bytes);
- `train(`x`)` perform a training using vectors x to prepare the quantizer;
- `compute_codes(`x`)` encode a n vectors to an array of size $n \times$(`code_size`);
- `decode(`C`)` decode the vectors from codes obtained with `encoder`;

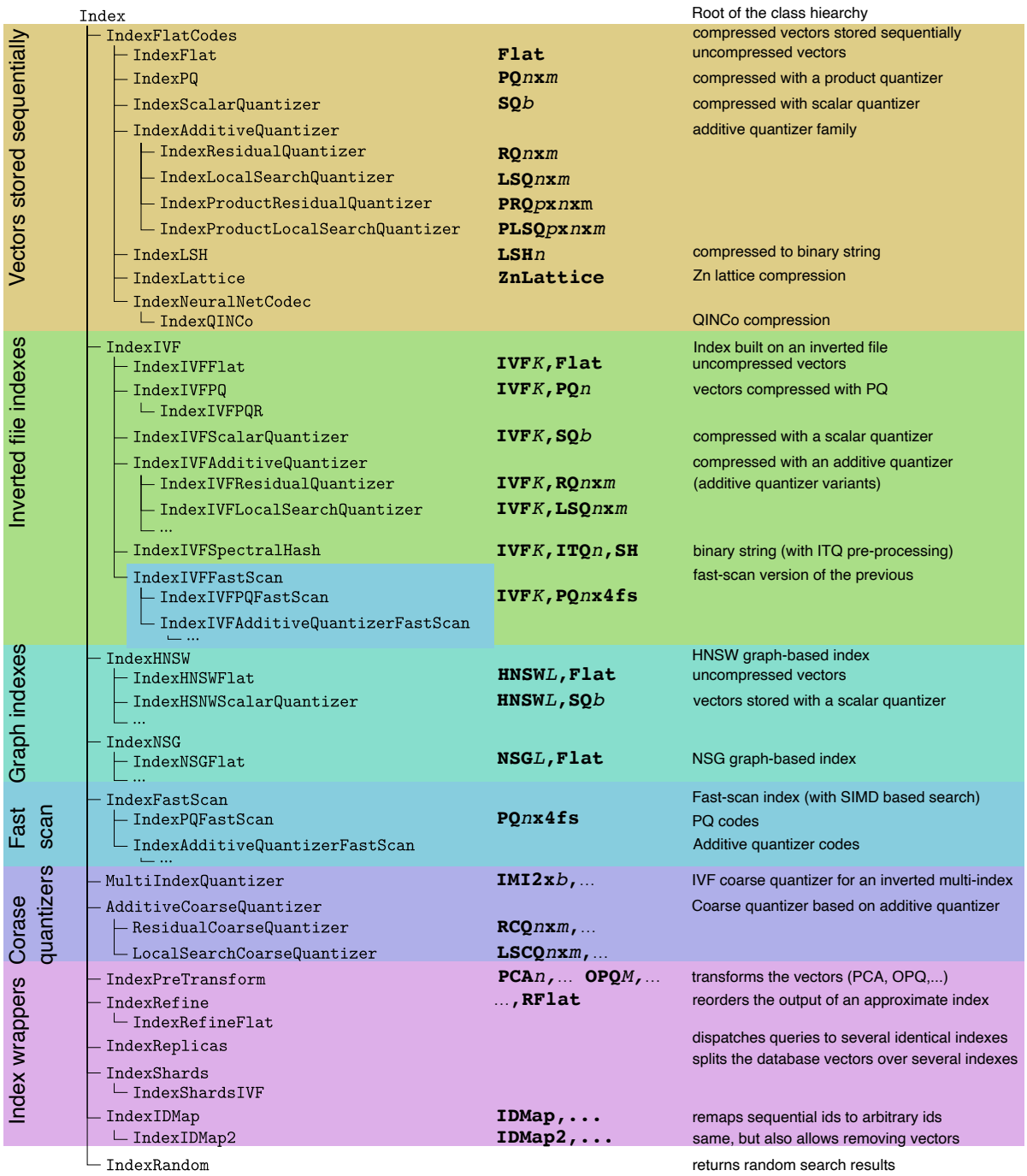


Figure 11: Hierarchy of the main index classes in CPU Faiss for floating-point vectors. For each class we indicate the corresponding factory string (when applicable) and a short explanation.

Table 3: Faiss metric types and corresponding function in `scipy.distance`. Note that the Faiss names are often more terse than their scipy counterparts.

Faiss name	scipy name	definition	comments
METRIC_INNER_PRODUCT		$\langle x, y \rangle$	
METRIC_L2	sqeuclidean	$\|x - y\|^2$	The Eucliean metric can be obtained via a square root
METRIC_L1	cityblock	$\sum_i \|x_i - y_i\|$	
METRIC_Linf	chebyshev	$\max_i \|x_i - y_i\|$	
METRIC_Lp	minkowski	$\sum_i \|x_i - y_i\|^p$	the parameter p is set via the metric_arg parameter
METRIC_Canberra	canberra	$\sum_i \|x_i - y_i\|/(\|x_i\| + \|y_i\|)$	
METRIC_BrayCurtis	braycurtis	$\sum_i \|x_i - y_i\|/\sum_i(\|x_i + y_i\|)$	
METRIC_JensenShannon	jensenshannon	$\frac{1}{2}(\mathrm{KL}(x\|\|m) + \mathrm{KL}(y\|\|m))$	where $m = \frac{1}{2}(x + y)$ and KL is the Kullback-Leiber divergence between two distributions
METRIC_Jaccard	jaccard	$\sum_i \min(x_i, y_i)/\sum_i \max(x_i, y_i)$	
METRIC_NaNEuclidean		$\frac{\|V\|}{d}\sum_{i \in V}(x_i - y_i)^2$	$V = \{i \in \{1..d\} \text{ st. } x_i \neq \mathrm{NaN} \wedge x_i \neq \mathrm{NaN}\}$

Table 4: A few combinations of pruning approaches (rows) and compression methods (columns). In the cells: the corresponding index implementations.

	No encoding	PQ encoding	scalar quantizer
Flat	IndexFlat	IndexPQ	IndexScalarQuantizer
IVF	IndexIVFFlat	IndexIVFPQ	IndexIVFScalarQuantizer
HNSW	IndexHSNWFlat	IndexHNSWPQ	IndexHNSWScalarQuantizer

Table 5: The hierarchy of quantizers. Each quantizer can represent the set of reproduction values of the quantizers below it.

Type of quantizer	class
Vector quantizer	
Additive quantizer	AdditiveQuantizer
Product-additive quantizer	ProductAdditiveQuantizer
Product quantizer	ProductQuantizer
Scalar quantizer	ScalarQuantizer
Binarization	